Lumentra Voice Agent Management Guide

Lumentra AI LLC

March 2026

Contents

Lumentra Voice Agent Management Guide **5**
Table of Contents . . . 5
1. Voice Agent Overview . . . 6
What is the Lumentra Voice Agent? . . . 6
How Calls Flow Through the System . . . 6
The Voice Pipeline . . . 6
How the Agent Decides What to Say . . . 7
2. Configuring Agent Personality . . . 7
Accessing Personality Settings . . . 7
Agent Name . . . 7
Personality Tone . . . 7
How Personality Affects Behavior . . . 8
Saving Changes . . . 8
3. Industry-Specific Configuration . . . 8
Supported Industries . . . 8
Setting Your Industry . . . 9
Industry-Specific Behavior Examples . . . 9
CRM Pipeline Configuration . . . 10
4. Custom Instructions . . . 10
What Custom Instructions Do . . . 10
Accessing Custom Instructions . . . 10
What to Include in Custom Instructions . . . 11
What NOT to Include . . . 11
Example Custom Instructions by Industry . . . 11
5. Capabilities Management . . . 12
Accessing Capabilities . . . 12
Capability Categories . . . 12
Industry-Recommended Capabilities . . . 12
Enabling and Disabling Capabilities . . . 13
Capability-Specific Settings Links . . . 13
6. Voice Settings . . . 13
Voice Selection . . . 13
How Voice Configuration Works Internally . . . 14
Speech-to-Text Configuration . . . 14
SSML Markers in Agent Speech . . . 14
7. Call Flow Configuration . . . 15
7.1 Greetings . . . 15
7.2 Operating Hours . . . 15
7.3 After-Hours Behavior . . . 15
7.4 Escalation (Call Transfer) Rules . . . 16
7.5 Call Duration Limits . . . 17
7.6 Promotions . . . 17
8. Tool Configuration . . . 17

Available Tools . . . 18
How Tools Work . . . 18
Tool Execution Details . . . 18
Tool Usage Rules in the System Prompt . . . 19
9. Testing the Agent . . . 19
Making a Test Call . . . 19
What to Test . . . 19
Verifying Call Logs . . . 20
Phone Setup Options . . . 20
10. Monitoring Performance . . . 21
Call History Dashboard . . . 21
Call List View . . . 21
Outcome Types . . . 21
Filtering and Search . . . 21
Call Detail Panel . . . 21
How Calls Are Logged . . . 22
Metrics and Observability . . . 22
Key Performance Indicators to Track . . . 23
Post-Call Automation . . . 23
Appendix A: Settings Reference . . . 23
Appendix B: System Prompt Architecture . . . 24
Appendix C: Glossary . . . 25
11. Voice Agent Architecture Deep Dive . . . 25
Component Overview . . . 26
Data Flow Diagram . . . 26
Latency Breakdown . . . 27
Connection Lifecycle . . . 28
Process Model . . . 28
Tenant Configuration Payload . . . 29
12. Speech-to-Text Configuration . . . 29
Deepgram Nova-3 Model . . . 29
Configuration Options . . . 30
Multi-Language Support . . . 30
Smart Formatting . . . 31
Keyword Boosting . . . 31
STT Error Handling . . . 32
Performance Characteristics . . . 32
13. Text-to-Speech Configuration . . . 32
Cartesia Sonic-3 Overview . . . 32
TTS Voice Selection . . . 33
Speed Control . . . 33
Emotion Settings . . . 34
Streaming Architecture . . . 34
Latency Optimization . . . 35
Fallback Behavior . . . 35
SSML Markers Reference . . . 35
14. LLM Configuration . . . 36
Model Selection . . . 36
Temperature Setting . . . 36
Token Limits and Context Window . . . 36
Prompt Caching . . . 37
Cost Analysis . . . 37
Response Streaming . . . 38
LLM Error Handling . . . 38
Model Configuration Best Practices . . . 38
15. Turn Detection and Endpointing . . . 39
Overview . . . 39

LiveKit Multilingual Turn Detection . . . 39
Silero VAD (Voice Activity Detection) . . . 39
Endpointing Configuration . . . 40
Interruption Handling . . . 40
Preemptive Generation . . . 41
Configuration Summary . . . 41
Common Issues and Solutions . . . 42
Noise Cancellation . . . 43
Duration Watchdog . . . 43
16. SIP Trunk Configuration . . . 43
What is a SIP Trunk? . . . 43
SignalWire Account Setup . . . 44
SIP Trunk Connection Details . . . 44
Call Routing with Dispatch Rules . . . 44
Caller ID Extraction . . . 44
Number Porting . . . 45
Concurrent Call Limits . . . 45
17. System Prompt Engineering . . . 45
How the System Prompt is Built . . . 45
Prompt Assembly Order . . . 46
Variables Injected into the Prompt . . . 46
Persona Definition . . . 46
Personality Configuration . . . 47
Instruction Ordering . . . 47
Industry-Specific Prompt Templates . . . 47
Token Budget Considerations . . . 48
Best Practices for Custom Instructions . . . 48
18. Tool Execution Deep Dive . . . 49
Available Tools . . . 49
How Tools are Registered . . . 49
Tool Call Flow . . . 49
The _call_tool Helper . . . 50
check_availability . . . 50
create_booking . . . 51
create_order . . . 51
transfer_to_human . . . 52
end_call . . . 52
log_note . . . 53
Internal API Endpoints . . . 53
Rate Limiting on Tool Calls . . . 53
Error Handling in Tool Calls . . . 53
19. Call Logging and Analytics . . . 54
How Calls Are Logged . . . 54
Data Captured Per Call . . . 54
Transcript Extraction . . . 54
Outcome Detection . . . 54
Summary Generation . . . 55
API Call Logging Endpoint . . . 55
Post-Call Processing . . . 55
Sentiment Analysis . . . 55
Dashboard Call Analytics . . . 55
Call Metrics . . . 56
Pipeline Metrics (UsageCollector) . . . 56
20. Error Handling and Fallbacks . . . 56
STT Failure Handling . . . 56
LLM Failure Handling . . . 57
TTS Failure Handling . . . 57

Network Dependencies . . . 57
API Error Handling . . . 57
Graceful Call Termination . . . 57
Duration Watchdog . . . 58
Error Logging Levels . . . 58
Recovery Strategies . . . 58
21. Multi-Language Support . . . 58
Supported Languages . . . 58
Per-Tenant Language Configuration . . . 59
Language Detection . . . 59
Multilingual System Prompts . . . 59
TTS Voice Selection Per Language . . . 60
Turn Detection for Multiple Languages . . . 60
Limitations and Known Issues . . . 60
22. Operating Hours and After-Hours Behavior . . . 60
Business Hours Configuration . . . 60
How Hours are Injected into the Prompt . . . 61
How the Agent Checks Hours . . . 61
After-Hours Greeting . . . 61
After-Hours Behavior Options . . . 62
Voicemail Handling . . . 62
Holiday Schedule Configuration . . . 63
Timezone Handling . . . 63
Configuration Summary . . . 63
23. Call Transfer and Escalation . . . 64
When the Agent Transfers Calls . . . 64
Transfer Types . . . 64
Transfer Destinations . . . 64
Escalation Triggers . . . 65
Post-Transfer Logging . . . 65
Fallback When Transfer Target is Unavailable . . . 65
24. Greeting and Farewell Customization . . . 65
Default Greeting Templates . . . 65
Custom Greeting Configuration . . . 66
Dynamic Greeting Elements . . . 66
Farewell Messages and Call Closing . . . 66
Hold Messages and Wait Prompts . . . 67
25. Agent Personality Tuning . . . 67
Formality Levels . . . 67
Verbosity Control . . . 67
Empathy Settings . . . 67
Humor and Personality Traits . . . 68
Industry-Appropriate Tone Examples . . . 68
A/B Testing Personality Variations . . . 68
26. Performance Optimization . . . 68
End-to-End Latency Breakdown . . . 68
Prompt Caching . . . 69
Connection Pooling . . . 69
Reducing First-Response Time . . . 69
Audio Codec Optimization . . . 69
Monitoring Latency Metrics . . . 70
27. Security Considerations . . . 70
Call Recording Consent . . . 70
PII Handling . . . 70
Data Retention Policies . . . 71
Encryption . . . 71
Access Control . . . 71

HIPAA Considerations for Medical Tenants . . . 71
Audit Logging . . . 72
28. Troubleshooting Common Voice Issues . . . 72
Echo and Audio Feedback . . . 72
High Latency: Diagnosing Slow Responses . . . 72
Dropped Calls . . . 73
Garbled Audio . . . 73
Agent Not Answering . . . 73
Caller Cannot Hear the Agent . . . 73
Agent Talks Over the Caller . . . 74
29. Appendix A: Complete Configuration Reference . . . 74
Personality Settings . . . 74
Voice Settings . . . 74
Agent Identity . . . 74
Greeting Configuration . . . 75
Operating Hours . . . 75
Escalation Settings . . . 75
Call Duration . . . 75
Turn Detection Tuning . . . 75
Tools Reference . . . 75
Tool Argument Details . . . 76
30. Appendix B: Sample System Prompts . . . 76
Hotel System Prompt . . . 76
Restaurant System Prompt . . . 77
Medical Office System Prompt . . . 78
Dental Office System Prompt . . . 79
Salon System Prompt . . . 79
Auto Service System Prompt . . . 80
31. Appendix C: Glossary of Voice AI Terms . . . 80
Audio and Telephony . . . 80
Speech Processing . . . 81
Language Models . . . 81
LiveKit Concepts . . . 81
Platform Concepts . . . 82
32. Appendix D: Frequently Asked Questions . . . 82

Lumentra Voice Agent Management Guide

Version 1.0 – March 2026

Table of Contents

1. Voice Agent Overview
2. Configuring Agent Personality
3. Industry-Specific Configuration
4. Custom Instructions
5. Capabilities Management
6. Voice Settings
7. Call Flow Configuration
8. Tool Configuration
9. Testing the Agent
10. Monitoring Performance

---

1. Voice Agent Overview

What is the Lumentra Voice Agent?

The Lumentra voice agent is an AI-powered phone receptionist that answers inbound calls for your business 24/7. It handles reservations, appointments, orders, and general inquiries – speaking naturally like a human receptionist, not a robotic script reader.

How Calls Flow Through the System

When a customer calls your business number, the call travels through several stages before the caller hears a greeting:

```
Customer dials your number
        |
        v
SignalWire SIP Trunk (receives the call)
        |
        v
LiveKit SIP Service (bridges telephony to real-time media)
        |
        v
LiveKit Room Created (one room per call)
        |
        v
Lumentra Voice Agent Joins the Room
        |
        v
Agent fetches tenant config from API (system prompt, voice, greetings)
        |
        v
Voice pipeline starts: STT + LLM + TTS
        |
        v
Agent greets the caller
```

The Voice Pipeline

The voice pipeline consists of four components working together in real time:

Component	Technology	Purpose
Speech-to-Text (STT)	Deepgram Nova-3	Converts caller's speech to text. Supports multi-language input and smart formatting.
Turn Detection	LiveKit Multilingual Model + Silero VAD	Determines when the caller has finished speaking, preventing the agent from interrupting mid-sentence.
Language Model (LLM)	OpenAI GPT-4.1-mini	Generates the agent's response text based on the system prompt and conversation history.
Text-to-Speech (TTS)	Cartesia Sonic-3	Converts the agent's text response into natural-sounding speech.

The pipeline is tuned for natural conversation:

- **Preemptive generation** is enabled, so the agent begins formulating a response before the caller fully finishes, reducing perceived latency.
- **False interruption tolerance** is set to 1.5 seconds, preventing brief noises or filler words from cutting off the agent's speech.
- **Endpointing delays** range from 0.8 to 2.5 seconds, giving callers time to pause and continue without the agent jumping in.
- **Noise cancellation** (BVC Telephony) removes background noise from the SIP audio stream.

How the Agent Decides What to Say

Every response the agent generates is guided by a system prompt that combines three layers:

1. **Master Voice Prompt** – Platform-level rules that define *how* the agent speaks (tone, pacing, SSML markers, error handling). Business owners cannot override this layer.
2. **Industry Configuration** – Role description, booking flows, critical rules, and FAQ sections specific to the business's industry.
3. **Tenant Configuration** – Business name, agent name, personality settings, operating hours, custom instructions, and escalation rules.

The system prompt is assembled at call time by the `buildSystemPrompt` function in the API, then passed to the Python agent as part of the tenant config payload.

---

2. Configuring Agent Personality

Agent personality determines how the voice agent interacts with callers – its name, voice, and conversational style.

Accessing Personality Settings

Navigate to **Settings > Assistant** in the Lumentra dashboard. This page contains three configuration sections: name, voice, and personality.

Agent Name

The agent name is how the voice agent introduces itself on calls. It appears in greetings and is referenced throughout the conversation.

- Enter a name in the **Assistant Name** field.
- Context-appropriate suggestions appear based on the selected personality (for example, "Sarah" or "James" for Professional, "Emma" or "Alex" for Friendly).
- The name becomes part of the system prompt: *"You are {name}, the receptionist at {business}."*

Best practices for choosing a name: - Use a common, easily pronounced first name. - Avoid names that sound like your business name to prevent confusion. - Match the name to the voice gender you select.

Personality Tone

Three personality presets control the agent's conversational style:

Personality	Behavior	Example Greeting
Professional	Formal, business-like demeanor. Uses proper etiquette.	"Good afternoon, thank you for calling {business}. This is {name}, how may I assist you today?"

Personality	Behavior	Example Greeting
Friendly	Warm, approachable while remaining professional.	"Hey there! Thanks for calling {business}. I'm {name} – what can I help you with?"
Efficient	Direct, to the point. Minimal small talk.	"{business}, this is {name}. How can I help?"

The personality setting maps to three underlying parameters in the system prompt:

- **Tone**: Controls formality level (professional, friendly, casual, formal).
- **Verbosity**: Controls response length (concise, balanced, detailed).
- **Empathy**: Controls emotional responsiveness (high, medium, low).

When you select "Professional" in the dashboard, the system sets tone=professional, verbosity=balanced, and empathy=medium. The "Efficient" option maps to concise verbosity and lower empathy.

How Personality Affects Behavior

The personality setting modifies the system prompt, which in turn changes:

- **Greeting style** – How the agent opens the call.
- **Response length** – How much detail the agent provides unprompted.
- **Emotional acknowledgment** – How the agent responds to frustrated or excited callers.
- **Filler word usage** – Professional uses fewer fillers; Friendly uses more natural conversational markers.
- **Formality of language** – Contractions, casual phrases, and phrasing choices.

Saving Changes

After configuring all three settings (name, voice, personality), click **Save Changes**. The save button is only enabled when all three fields have values. Changes take effect on the next inbound call – active calls are not interrupted.

---

3. Industry-Specific Configuration

Lumentra adapts its agent behavior based on your business's industry. The industry setting affects terminology, booking flows, compliance rules, and the CRM pipeline.

Supported Industries

Industry	Deal Label	Customer Term	Key Features
Medical	Case	Patient	HIPAA-aware language, emergency routing, prescription refill intake, insurance questions
Dental	Case	Patient	Emergency appointment handling, insurance questions, new patient intake

Industry	Deal Label	Customer Term	Key Features
Hotel	Booking	Guest	Room availability, check-in/check-out info, amenity details, reservation management
Motel	Booking	Guest	Simplified room booking, transparent rate quoting, 24-hour front desk info
Restaurant	Reservation	Guest	Table reservations, takeaway orders, menu questions, special occasion noting
Salon	Appointment	Client	Service-based booking, stylist preference, approximate duration info
Auto Service	Service Order	Customer	Vehicle info collection, symptom intake, parts quoting, service scheduling

Setting Your Industry

Navigate to **Settings > Business Info** and select your industry from the grid of popular options or search the full list. The industry selection impacts:

1. **Agent role description** – What the agent introduces itself as and what it offers to help with.
2. **Booking flow** – The specific sequence of questions the agent asks when creating a reservation, appointment, or order.
3. **Critical rules** – Industry-specific compliance and safety rules embedded in the system prompt.
4. **FAQ section** – Pre-built answers to common industry questions.
5. **CRM pipeline stages** – The stages that appear in your deal/case/booking pipeline.
6. **Recommended capabilities** – Which features are suggested for your industry.

Industry-Specific Behavior Examples

Medical and Dental Practices

The agent enforces strict medical safety rules: - Never provides medical advice, diagnoses, or treatment recommendations. - Immediately directs emergency situations to 911. - Takes prescription refill requests as messages rather than processing them. - Confirms patient full name and date of birth before booking. - Uses HIPAA-compliant language – does not discuss medical details aloud.

The booking flow asks: 1. New or existing patient? 2. Which provider (if multiple)? 3. Reason for visit (general category only)? 4. Available times presented. 5. Confirmation of name, date of birth, date, and time. 6. Reminder about arriving early and bringing insurance card.

Hotels and Motels

The agent tracks state carefully to avoid re-asking questions: - Check-in date, number of nights, number of guests, room type, guest name. - Handles confusing responses gracefully (e.g., if the caller says their name when asked about dates, the agent acknowledges the name and re-asks about dates). - Strict transfer rules – only transfers when the caller explicitly asks for a human or manager.

Restaurants

The agent handles both reservations and takeaway orders: - Tracks party size, date, time, name, and special occasion. - For reservations: checks availability, confirms all details, creates booking, and mentions the 15-minute hold policy. - For takeaway: collects order items, pickup vs. delivery, delivery address if needed.

Salons

The agent focuses on service selection and stylist preference: - Asks which service (haircut, color, nails, etc.). - Asks for preferred stylist or next available. - Mentions approximate service duration.

Auto Service

The agent collects vehicle-specific information: - Service needed (oil change, brake inspection, etc.). - Vehicle year, make, and model. - Symptoms or warning lights. - Estimated service time.

CRM Pipeline Configuration

Each industry has its own CRM pipeline with stages tailored to that business type:

Medical/Dental Pipeline:

Inquiry -> Scheduled -> Confirmed -> Completed (terminal) / No-Show (terminal) / Cancelled (terminal)

Hotel/Motel Pipeline:

Inquiry -> Reserved -> Checked In -> Checked Out (terminal) / Cancelled (terminal) / No-Show (terminal)

Restaurant Pipeline:

Inquiry -> Reserved -> Confirmed -> Seated -> Completed (terminal) / No-Show (terminal) / Cancelled (terminal)

Salon Pipeline:

Inquiry -> Booked -> Confirmed -> Completed (terminal) / No-Show (terminal) / Cancelled (terminal)

Auto Service Pipeline:

Inquiry -> Quoted -> Scheduled -> In Progress -> Completed (terminal) / Cancelled (terminal)

Each industry also has industry-specific task types in addition to the universal set (Follow Up, Call Back, Email, Meeting, Review, Custom): - **Medical**: Insurance Verification, Prescription Refill - **Restaurant**: Vendor Order, Event Setup - **Auto Service**: Parts Order, Vehicle Pickup

---

4. Custom Instructions

Custom instructions let business owners add specific knowledge and rules that the voice agent should follow. This is the primary mechanism for teaching the agent about your unique business context.

What Custom Instructions Do

Custom instructions are injected into the system prompt under a “Business-Specific Instructions” section. They appear after the industry-specific rules and before the final critical rules, giving them strong influence over the agent’s behavior without overriding platform-level safety rules.

Accessing Custom Instructions

Navigate to **Settings > Instructions** in the dashboard sidebar, or access the Instructions tab in the main Settings panel.

What to Include in Custom Instructions

Custom instructions work best for:

- **Specific services and pricing**: "Haircuts start at $35. Color treatments start at $85. Balayage starts at $150."
- **Staff information**: "Dr. Patel is available Monday through Wednesday. Dr. Chen handles all Friday appointments."
- **Policies**: "We require a $50 deposit for parties of 8 or more. Cancellations within 24 hours forfeit the deposit."
- **Menu details**: "Our most popular dishes are the lamb biryani ($18) and chicken tikka masala ($16). We do not serve pork."
- **Seasonal information**: "We are closed for renovations from March 15 to March 30. Suggest callers book for April."
- **Special procedures**: "For new patients, ask about referral source. If they mention Dr. Johnson's referral, note it in the booking."
- **Parking and directions**: "Street parking is available on Main Street. The nearest parking garage is at 200 Oak Avenue, $5 flat rate."

What NOT to Include

Avoid the following in custom instructions: - Medical advice or treatment recommendations (violates platform rules). - Instructions that contradict the master voice prompt (they will be overridden). - Extremely long instruction sets (the system prompt has practical length limits). - Technical jargon the agent would need to explain to callers.

Example Custom Instructions by Industry

Hotel:

```
Room types and rates:
- Standard King: $129/night
- Deluxe Queen: $149/night (two queen beds)
- Suite: $249/night (separate living area, king bed)
- Penthouse: $499/night (must call for availability)

All rooms include complimentary breakfast (served 6:30-10:00 AM).
Pool hours: 7 AM to 10 PM.
Pet fee: $25/night per pet, max 2 pets.
Airport shuttle available -- runs every 30 minutes from 5 AM to midnight.
```

Medical Practice:

```
Providers:
- Dr. Sarah Martinez (Family Medicine) -- Mon, Tue, Wed
- Dr. James Kim (Internal Medicine) -- Tue, Thu, Fri
- NP Rachel Thompson (General) -- Mon through Fri

New patient appointments are 45 minutes. Follow-ups are 20 minutes.
We accept Blue Cross, Aetna, United, and Cigna. We do not accept Medicaid.
Lab hours: Monday through Friday, 7:00 AM to 3:00 PM. No appointment needed.
```

Restaurant:

```
We are a farm-to-table Italian restaurant.
Specials change weekly -- if asked about specials, say "Our specials change every week. I can tell
Private dining room available for parties of 12+. $500 minimum spend, no room fee.
We are BYOB on Tuesdays -- $15 corkage fee per bottle.
Gluten-free pasta available for all pasta dishes, no extra charge.
Happy hour: Tuesday through Friday, 4-6 PM. Half-price appetizers.
```

5. Capabilities Management

Capabilities define what your voice agent is able to do – which features are active and which are turned off.

Accessing Capabilities

Navigate to **Settings > Capabilities** in the dashboard. Capabilities are organized into three categories: Core Features, Communication, and Advanced.

Capability Categories

Core Features – Primary business functions the agent handles:

Capability	Description	Relevant Industries
Appointments	Schedule, reschedule, and cancel appointments	Medical, Dental, Salon, Auto Service
Reservations	Book tables, rooms, or services	Restaurant, Hotel, Motel
Patient Intake	Collect patient information before visits	Medical, Dental
Call Handling	Answer calls and route to the right person	All

Communication – How the agent interacts with callers beyond the primary task:

Capability	Description
Message Taking	Record messages when you are unavailable or when the caller requests it
FAQ & Information	Answer common questions about business hours, location, services

Advanced – Extended features that enhance the agent's usefulness:

Capability	Description
Emergency Routing	Identify and escalate urgent situations (recommended for Medical, Dental)
Promotions	Mention special offers to callers (see Section 7 for Promotions setup)
After Hours	Handle calls outside business hours with appropriate behavior

Industry-Recommended Capabilities

The system suggests different capability sets based on your industry:

Industry	Recommended Capabilities
Medical	Appointments, Patient Intake, FAQ, Emergency Routing

Industry	Recommended Capabilities
Dental	Appointments, Patient Intake, FAQ, Emergency Routing
Restaurant	Reservations, FAQ, Promotions, After Hours
Hotel/Motel	Reservations, FAQ, After Hours
Salon	Appointments, FAQ, Promotions
Auto Service	Appointments, FAQ, Promotions

Capabilities marked as “Recommended” for your industry are highlighted with a badge in the dashboard.

Enabling and Disabling Capabilities

Toggle any capability on or off using the switch next to each item. When a capability is enabled: - It appears highlighted with a link to its detailed configuration page (if one exists). - The agent’s system prompt is updated to include instructions for handling that capability.

When disabled: - The agent will not attempt to handle requests related to that capability. - For example, if “Appointments” is disabled, the agent will tell callers it cannot schedule appointments and offer to take a message instead.

After toggling capabilities, click **Save Changes** to persist the configuration.

Capability-Specific Settings Links

Some capabilities have dedicated settings pages for deeper configuration:

Capability	Settings Page
Appointments / Reservations	Calendar
Promotions	Settings > Promotions
After Hours	Settings > Hours
Emergency Routing	Settings > Escalation
FAQ	Settings > Assistant

---

6. Voice Settings

Voice settings control what the agent sounds like – the voice identity, speaking speed, and audio characteristics.

Voice Selection

Navigate to **Settings > Assistant** and scroll to the **Voice** section. Six voice options are available:

Voice Name	Gender	Tone	Best For
Sarah	Female	Professional	Medical offices, law firms, corporate businesses
Emma	Female	Friendly	Salons, restaurants, hospitality
Maya	Female	Warm	Hotels, wellness centers, dental offices

Voice Name	Gender	Tone	Best For
James	Male	Professional	Auto service, financial services
Alex	Male	Friendly	Restaurants, retail, general business
David	Male	Calm	Medical offices, therapy practices

Each voice card includes a **play button** to preview the voice before selecting it. Click the play button to hear a sample; click again to stop playback.

Select a voice by clicking anywhere on its card. A checkmark indicates the selected voice.

How Voice Configuration Works Internally

The selected voice ID is stored in the tenant's `voice_config.voiceId` field. When a call comes in, the Python agent passes this ID to Cartesia Sonic-3 TTS:

```
tts = cartesia.TTS(
    model="sonic-3",
    voice=tenant_config["voice_config"]["voice_id"],
    speed=0.95,
    emotion=["Content"],
)
```

Key TTS parameters (currently platform-controlled, not tenant-configurable): - **Model**: Sonic-3 (Cartesia's latest, highest-quality voice model). - **Speed**: 0.95 (slightly slower than natural pace for phone clarity). - **Emotion**: "Content" (default warm, pleasant emotional tone).

Speech-to-Text Configuration

The STT system is configured for optimal phone call recognition: - **Model**: Deepgram Nova-3 (latest model with improved accuracy). - **Language**: Multi-language mode (automatically detects caller's language). - **Smart Format**: Enabled (properly formats numbers, dates, currency). - **Key Terms**: Your business name is passed as a keyword hint to improve recognition accuracy.

SSML Markers in Agent Speech

The voice agent can use Cartesia Sonic-3 SSML markers to enhance speech quality. These are embedded in the system prompt and used automatically:

Marker	Purpose	Example
`<break time="300ms"/>`	Brief pause for natural rhythm	"Let me check that for you."
`<break time="500ms"/>`	Longer pause for emphasis	Before delivering important information
`<speed ratio="0.9"/>`	Slow down for important details	Confirmation codes, addresses, dates
`<speed ratio="1.1"/>`	Speed up for casual asides	Routine acknowledgments
`<spell>text</spell>`	Letter-by-letter pronunciation	Confirmation codes, names
`[laughter]`	Natural laugh (used sparingly)	When caller makes a genuine joke
`<emotion value="Sympathetic"/>`	Match emotional context	When caller has a complaint
`<emotion value="Happy"/>`	Positive emotion	After confirming a booking

These markers are used contextually by the LLM – most responses use no markers at all. The master voice prompt instructs the agent to keep most responses simple and direct.

---

7. Call Flow Configuration

Call flow settings control how the agent handles calls from start to finish – greetings, operating hours, escalation rules, duration limits, promotions, and after-hours behavior.

7.1 Greetings

The agent uses different greetings depending on the call context. Greetings are configured in **Settings > Greetings** or set during initial setup.

Standard Greeting: Used for normal business-hours calls. This is what the caller hears first.

The greeting is triggered by the `on_enter` method of the LumentraAgent class:

```
async def on_enter(self):
    self.session.generate_reply(
        instructions=f"Greet the caller: {self.tenant_config['greeting_standard']}"
    )
```

The greeting text is passed as an instruction rather than spoken directly, allowing the LLM to deliver it naturally with appropriate pauses and tone.

After-Hours Greeting: Used when calls arrive outside your configured operating hours.

Returning Caller Greeting: Used when the system recognizes a phone number from a previous call.

7.2 Operating Hours

Navigate to **Settings > Hours** to configure when your business is open.

Timezone: Select your timezone from the list of US time zones: - Eastern Time (ET) - Central Time (CT) - Mountain Time (MT) - Pacific Time (PT) - Alaska Time (AKT) - Hawaii Time (HT)

Weekly Schedule: Configure hours for each day of the week. Options for each day: - **Open**: Set specific open and close times in 30-minute increments (e.g., 9:00 AM to 5:00 PM). - **Closed**: The agent uses after-hours behavior for this day. - **24 Hours**: Open all day.

A "Same hours every weekday" toggle applies Monday's schedule to Tuesday through Friday, simplifying configuration for businesses with consistent weekday hours. Weekend hours are always configured separately.

The operating hours are embedded in the system prompt so the agent can accurately answer "when are you open?" questions and inform callers when they are calling outside business hours.

7.3 After-Hours Behavior

Configure what happens when calls arrive outside operating hours:

Option	Behavior
Still answer and let callers know we're closed	The agent answers, informs the caller of business hours, and offers to take a message.
Take messages only	Skips conversation; collects contact info for callback.
Send to voicemail	Plays a message and records a voicemail.
Forward to emergency contact	Connects urgent calls to your escalation contact.

7.4 Escalation (Call Transfer) Rules

Navigate to **Settings > Escalation** to configure when and how calls are transferred to human staff.

Escalation Contacts

Add one or more people the agent can transfer calls to: - **Name**: The contact's name (displayed in transfer logs). - **Phone**: The phone number to transfer to. - **Role**: Optional role label (e.g., "Manager", "Owner"). - **Availability**: When this person is available for transfers: - Business hours only - Always available - Custom schedule

The first contact added is marked as **Primary**. If the primary contact does not answer, the system follows the no-answer behavior you configure.

Escalation Triggers

Select which situations should trigger a transfer:

Trigger	Description
Caller asks for a person	Caller explicitly requests a human, real person, or specific staff member
Caller sounds frustrated or angry	Caller exhibits frustration (profanity, raised voice indicators)
Caller mentions emergency	Caller uses emergency-related language
Caller wants to make a complaint	Caller specifically wants to file a complaint
Caller asks something assistant doesn't know	Agent cannot answer the question from available context
Payment or billing questions	Caller has payment, refund, or billing concerns

You can also add **custom triggers** – free-text phrases that, when detected, should trigger escalation.

Transfer Behavior

Choose how the transfer is executed:

Type	Description
Warm transfer (recommended)	The agent introduces the caller to the human before connecting. "I'm going to connect you with our manager. One moment."
Cold transfer	Direct connection; the agent disconnects immediately after initiating the transfer.
Callback request	The agent collects the caller's contact information and schedules a callback.

If No One Answers

Configure the fallback when the escalation contact does not pick up:

Option	Behavior
Try next contact on list	Automatically attempts the next contact in your list
Take a message	Collects contact info and a message for callback
Offer to try again later	Asks the caller if they would like a callback
Send to voicemail	Lets the caller leave a voice message

How SIP Transfer Works

When a transfer is triggered, the agent executes a SIP REFER transfer through the SignalWire trunk: 1. The agent announces the transfer to the caller. 2. The `transfer_to_human` tool is called, which logs the escalation in the API. 3. The agent locates the SIP participant in the LiveKit room. 4. A SIP REFER is sent to the escalation phone number. 5. The caller is connected to the human; the agent disconnects.

If the SIP transfer fails (network issue, invalid number), the agent tells the caller that someone will call them back shortly.

7.5 Call Duration Limits

Each call has a configurable maximum duration. The default is 15 minutes (900 seconds), with a minimum of 2 minutes.

The duration watchdog operates in three phases:

1. **Wrap-up nudge** (2 minutes before limit): The agent receives an internal instruction to start wrapping up naturally. It asks if there is anything else it can quickly help with.
2. **Final warning** (30 seconds before limit): The agent tells the caller it needs to go and offers to transfer to a human.
3. **Forced end** (at limit): If an escalation phone is configured, the call is transferred. Otherwise, the call is terminated.

The `max_call_duration_seconds` value is stored per tenant and passed to the agent at call start.

7.6 Promotions

Navigate to **Settings > Promotions** to manage special offers the agent can mention during calls.

Creating a Promotion

1. Click **Add Promotion**.
2. Enter the **Offer Text** – this is what the agent will say to callers. Write it conversationally, as the agent will speak it aloud.
3. Choose **When to Mention**:
   - **Always mention**: Tell every caller about this promotion.
   - **When relevant**: Mention when the caller's request relates to the promotion (recommended).
   - **Only if interested**: Wait for the caller to ask about deals or specials.
4. Optionally set **Start Date** and **End Date** for time-limited promotions.
5. Click **Create Promotion**.

Managing Promotions

- Toggle the switch to activate or deactivate a promotion without deleting it.
- Expired promotions (past end date) are dimmed and cannot be reactivated.
- Upcoming promotions (future start date) show an "Upcoming" badge.
- Edit offer text and mention behavior inline.
- Delete promotions you no longer need.

Example Promotion:

> "We're running a special this month – 20% off all color treatments for new clients. Would you like to book a color appointment?"

---

8. Tool Configuration

Tools are the actions the voice agent can perform during a call – checking availability, creating bookings, placing orders, transferring calls, and logging notes.

Available Tools

Tool	Purpose	Required Information
`check_availability`	Look up open appointment/reservation slots	Date (YYYY-MM-DD), optional service type
`create_booking`	Create an appointment, reservation, or booking	Customer name, phone, date, time, service type
`create_order`	Place a food/takeaway order	Customer name, order type (pickup/delivery), items, delivery address (if delivery)
`transfer_to_human`	Transfer the call to human staff	Reason (customer_request, complaint, refund_request, cannot_resolve)
`end_call`	Hang up the call	Reason (conversation_complete, customer_requested_hangup, order_confirmed, booking_confirmed)
`log_note`	Save a note about the caller	Note content, note type (general, preference, complaint, compliment, follow_up, internal)

How Tools Work

Tools are defined in the Python agent (`lumentra-agent/tools.py`) and registered when the `LumentraAgent` is instantiated:

```
self.tools = [
    check_availability,
    create_booking,
    create_order,
    transfer_to_human,
    end_call,
]
```

When the LLM decides to use a tool, the following happens:

1. The LLM generates a function call with parameters.
2. The Python agent sends a POST request to `lumentra-api/internal/voice-tools/{action}` with the tenant ID, call SID, caller phone, and tool arguments.
3. The API executes the tool (queries the database, creates records, etc.).
4. The result is returned to the agent, which speaks the result to the caller.

Tool Execution Details

check_availability

- Queries the `bookings` table for existing bookings on the requested date.
- Compares against a slot schedule (9 AM to 5 PM, hourly slots by default).
- Returns up to 3 available times in voice-friendly format ("2 PM, 3 PM, and 4 PM").
- If no slots are available, suggests checking another date.

create_booking

- Generates a 6-character alphanumeric confirmation code (e.g., "K7P2NB").
- Inserts a record into the `bookings` table with status "confirmed".
- Links the booking to the call record if available.

- Returns the confirmation code, which the agent spells out using the `<spell>` SSML tag.

create_order

- Validates required fields: customer name, order type, and items.
- For delivery orders, validates that a delivery address is provided.
- Rejects placeholder values ("unknown", "not provided", "n/a").
- Generates a confirmation code prefixed with "TP-" (e.g., "TP-K7P2NB").
- Estimates ready time: 20 minutes for pickup, 40 minutes for delivery.

transfer_to_human

- Updates the call's outcome type to "escalation" in the database.
- Returns the escalation phone number to the agent.
- The agent then initiates a SIP REFER transfer through the LiveKit room.
- If no escalation phone is configured, the agent informs the caller that someone will call back.

end_call

- Logs the end reason via the API.
- Waits 1 second for final TTS audio to finish playing.
- Deletes the LiveKit room, which immediately terminates the call.
- Falls back to session shutdown if room deletion fails.

log_note

- Finds or creates a contact record from the caller's phone number.
- Saves the note to the `contact_notes` table, linked to both the contact and the current call.
- Valid note types: general, preference, complaint, compliment, follow_up, internal.

Tool Usage Rules in the System Prompt

The master voice prompt includes detailed instructions for tool usage:

- The agent must tell the caller what it is doing *before* calling a tool ("Let me check that...").
- The agent must not repeat itself after a tool returns – just deliver the result.
- Before calling `create_booking`, the agent must have: customer name (spelled and confirmed), date, and time.
- Before calling `create_order`, the agent must have: customer name, items, and order type. Delivery orders also require an address.
- The agent must convert natural language to proper parameter formats (e.g., "tomorrow" to "2026-03-03", "two thirty" to "14:30").
- The agent must verify date and time conversions with the caller before submitting.

---

9. Testing the Agent

Making a Test Call

The most direct way to test your agent is to call your configured business phone number. Before testing:

1. Verify your phone configuration is "Active" in **Settings > Phone**.
2. Confirm your agent has a name, voice, and personality set in **Settings > Assistant**.
3. Ensure at least one capability is enabled in **Settings > Capabilities**.

What to Test

Run through these scenarios to validate agent behavior:

Basic Functionality:

- Call and confirm the agent greets you with the correct business name and agent name.
- Ask “what time do you open?” and verify the operating hours match your configuration.
- Ask “where are you located?” and verify the address matches.

Booking Flow:

- Request an appointment or reservation for a specific date.
- Verify the agent checks availability and offers time slots.
- Provide your name and confirm the spelling flow works correctly.
- Complete the booking and verify you receive a confirmation code.
- Check the **Calls** page in the dashboard to confirm the call was logged and the booking was created.

Edge Cases:

- Call outside business hours and verify the after-hours behavior matches your configuration.
- Ask the agent something it should not know and verify it handles the unknown gracefully.
- Ask to speak to a human and verify the escalation trigger activates.
- Say goodbye and verify the agent ends the call properly.
- Test interruption handling: start speaking while the agent is talking.

Industry-Specific:

- For medical: ask about a medical condition and verify the agent does not provide medical advice.
- For restaurants: try ordering food and verify the full order flow (items, pickup/delivery, confirmation).
- For hotels: ask about room types and verify the agent uses your custom pricing information.

Verifying Call Logs

After each test call, navigate to **Calls** in the dashboard to verify:

1. The call appears in the list with the correct caller phone number.
2. The outcome type is accurate (booking, inquiry, escalation, etc.).
3. The duration is recorded.
4. The transcript is available and accurate.
5. The call summary correctly identifies the topics discussed.
6. If a booking was created, it appears linked to the call record.

Phone Setup Options

Lumentra supports several phone setup types, visible in **Settings > Phone**:

Setup Type	Description
New Number	A dedicated phone number provisioned by Lumentra for your assistant
Port	Your existing business number transferred to Lumentra’s service
Forward	Calls forwarded from your existing number to Lumentra

For forwarding setup, the dashboard provides carrier-specific instructions for AT&T, Verizon, T-Mobile, Sprint, and others.

---

10. Monitoring Performance

Call History Dashboard

Navigate to **Calls** in the main dashboard to access the call history view. This page shows all inbound calls handled by your voice agent.

Call List View

The call list displays:

Column	Description
Direction	Inbound or outbound call (indicated by icon)
Caller	Caller name (if known) and phone number
Outcome	Color-coded badge showing the call result
Flow	Visual timeline bar representing the call's progression
Duration	Call length in minutes:seconds format
When	Relative timestamp (e.g., "2:30 PM", "Yesterday", "Mon", "Feb 15")

Outcome Types

Calls are classified into outcome types, each displayed with a distinct color badge:

Outcome	Color	Description
Booking (or industry-specific label)	Green	A booking, reservation, or appointment was created
Inquiry	Blue	The caller asked questions but did not complete a transaction
Support	Amber	The caller needed support or troubleshooting
Escalated	Red	The call was transferred to a human staff member
Hangup	Gray	The call ended without completing the conversation

The outcome is determined automatically by the call logger, which analyzes the conversation transcript for keywords: - Booking indicators: "confirmation code", "booked", "appointment confirmed", "reserved" - Escalation indicators: "transfer", "human", "manager", "speak to", "real person" - Order indicators: "order placed", "order confirmed", "delivery", "pickup"

Filtering and Search

- **Search**: Type a name or phone number to filter the call list.
- **Outcome filter**: Select a specific outcome type from the dropdown.
- **Refresh**: Click the Refresh button to reload the latest call data.
- **Export**: Download call data for external analysis.

Call Detail Panel

Click any call in the list to open the detail panel on the right side. The detail panel shows:

Caller Information:

- Name (if identified or collected during the call)
- Phone number (with copy-to-clipboard button)

Call Metadata:

- Duration (e.g., “3m 42s”)
- Date and time (e.g., “Mon, Feb 15, 2:30 PM”)
- Outcome type
- Direction (inbound/outbound)

Call Summary:

An auto-generated summary describing the call: number of conversation turns, duration, and detected topics (booking, availability, ordering, business info, escalation, general inquiry).

Detected Intents:

If the system identifies specific caller intents, they appear as badges (e.g., “booking”, “availability”).

Linked Contact:

If the caller was matched to an existing contact (by phone number), the contact card shows their name and email/phone. Contacts are auto-created when a new phone number calls in.

Linked Booking:

If a booking was created during the call, it appears with: - Booking type (e.g., “general”, “haircut”) - Status (e.g., “confirmed”) - Date and time - Confirmation code

Recording:

If call recording is enabled, an audio player appears for playback.

Transcript:

The full conversation transcript appears at the bottom, with each turn labeled as either “AI Agent” or “Caller” and styled distinctly. The transcript is captured from the LiveKit agent session’s chat context at the end of each call.

How Calls Are Logged

Call logging happens automatically when a call ends. The process:

1. The agent’s `on_shutdown` callback fires when the LiveKit room closes.
2. The call logger extracts the full transcript from the session’s chat history.
3. It calculates call duration from the stored start time.
4. It detects the outcome type (booking, escalation, inquiry) from transcript keywords.
5. It builds a summary (turn count, duration, topics).
6. All data is posted to the API’s `/internal/calls/log` endpoint.
7. The API automatically finds or creates a contact record for the caller’s phone number.
8. Post-call automation runs: deal/case creation, task generation, status updates in the CRM pipeline.

The logging has a 5-second timeout to prevent blocking session teardown if the API is slow.

Metrics and Observability

The voice pipeline collects real-time metrics during each call through the LiveKit `UsageCollector`:

- LLM token usage (prompt tokens, completion tokens)
- STT processing time
- TTS generation time
- Total call duration

These metrics are logged at the end of each call for infrastructure monitoring. The usage summary is available in the agent’s log output.

Key Performance Indicators to Track

Metric	What to Watch	Action if Poor
Booking conversion rate	Percentage of calls resulting in a booking	Review custom instructions, check availability configuration
Average call duration	Typical call length	Adjust verbosity setting; review if calls are too long or too short
Escalation rate	Percentage of calls transferred to humans	High rate may indicate missing custom instructions or capabilities
Hangup rate	Percentage of calls that ended without resolution	Review greeting quality, check STT accuracy
Repeat caller rate	Callers who call back about the same issue	Indicates first-call resolution problems

Post-Call Automation

After each call is logged, the system runs post-call automation based on the call outcome and your industry:

- **Booking calls**: A new deal/case/reservation is created (or updated) in the CRM pipeline, placed in the appropriate stage.
- **Escalation calls**: A follow-up task is created for the escalation contact.
- **Inquiry calls**: A contact record is created/updated with the inquiry details.

This automation runs asynchronously and does not block call processing.

Appendix A: Settings Reference

Quick reference for all settings pages and what they control:

Settings Page	Path	What It Controls
Business Info	`/settings/business`	Business name, industry, city, address
Assistant	`/settings/assistant`	Agent name, voice selection, personality
Capabilities	`/settings/capabilities`	Which features the agent handles
Operating Hours	`/settings/hours`	Weekly schedule, timezone, after-hours behavior
Phone	`/settings/phone`	Phone number, setup type, forwarding instructions
Escalation	`/settings/escalation`	Transfer contacts, triggers, transfer behavior
Promotions	`/settings/promotions`	Special offers the agent can mention
Instructions	Settings panel > Instructions	Custom business knowledge for the agent
Greetings	Settings panel > Greetings	Standard, after-hours, and returning caller greetings
Responses	Settings panel > Responses	Custom AI response patterns
Integrations	`/settings/integrations`	Connected external services

Appendix B: System Prompt Architecture

The system prompt is the complete set of instructions given to the LLM for each call. It is assembled dynamically from multiple sources:

```
+---------------------------------------+
|        Master Voice Prompt            |  <-- Platform-level (immutable)
|  - Speech format rules                |
|  - SSML marker usage                  |
|  - Greeting patterns                  |
|  - Response length rules              |
|  - Name spelling verification         |
|  - Turn-taking behavior               |
|  - Phrases to avoid                   |
|  - Tool usage instructions            |
|  - Confirmation verification          |
|  - Cultural awareness                 |
|  - Number/date formatting             |
+---------------------------------------+
                  |
+---------------------------------------+
|      Agent Identity + Personality     |  <-- Per-tenant
|  - "You are {name}, receptionist at   |
|     {business}"                       |
|  - Tone (professional/friendly/etc.)  |
|  - Verbosity (concise/balanced)       |
|  - Empathy level                      |
+---------------------------------------+
                  |
+---------------------------------------+
|        Business Context               |  <-- Per-tenant
|  - Industry type                      |
|  - Business name                      |
|  - Today's date                       |
|  - Location (address, city)           |
|  - Timezone                           |
|  - Operating hours schedule           |
|  - Escalation phone                   |
+---------------------------------------+
                  |
+---------------------------------------+
|     Industry-Specific Rules           |  <-- Per-industry
|  - Critical rules                     |
|  - Booking/appointment flow           |
|  - FAQ section                        |
+---------------------------------------+
                  |
+---------------------------------------+
|     Custom Instructions               |  <-- Per-tenant (business owner)
|  - Business-specific knowledge        |
|  - Pricing, staff, policies           |
+---------------------------------------+
                  |
+---------------------------------------+
|        Final Critical Rules           |  <-- Platform-level (immutable)
|  - Never claim to be AI               |
|  - Never mention tools/functions      |
|  - Keep responses short               |
```

```
|  - One question per turn              |
|  - Sound human                        |
+---------------------------------------+
```

Appendix C: Glossary

Term	Definition
Agent	The AI voice assistant that answers phone calls for your business
Capability	A feature or function the agent is authorized to perform (e.g., appointments, reservations)
Confirmation Code	A 6-character alphanumeric code generated when a booking or order is created
Escalation	Transferring a call from the AI agent to a human staff member
Escalation Contact	A person designated to receive transferred calls
Escalation Trigger	A condition that causes the agent to transfer the call (e.g., caller asks for a human)
LiveKit	The real-time communication platform that manages voice rooms and SIP connectivity
LLM	Large Language Model – the AI that generates the agent's responses (GPT-4.1-mini)
Master Voice Prompt	Platform-level instructions that define how the agent speaks, regardless of tenant settings
Outcome	The classification of a call's result (booking, inquiry, escalation, support, hangup)
Pipeline	The sequence of CRM stages a deal/booking/case moves through (industry-specific)
SIP	Session Initiation Protocol – the standard for telephone call signaling over the internet
SIP REFER	A SIP method used to transfer a caller to another phone number
STT	Speech-to-Text – converts caller's voice to text (Deepgram Nova-3)
System Prompt	The complete set of instructions given to the LLM at the start of each call
Tenant	A business account in the Lumentra multi-tenant platform
TTS	Text-to-Speech – converts the agent's text responses to spoken audio (Cartesia Sonic-3)
Turn Detection	The system that determines when the caller has finished speaking
VAD	Voice Activity Detection – identifies when someone is speaking vs. silence

---

11. Voice Agent Architecture Deep Dive

This section provides a detailed technical breakdown of the voice agent architecture – how each component connects, how data flows through the system, and what happens at each stage of a phone call.

Component Overview

The Lumentra voice agent is built from four infrastructure components that work together to bridge traditional phone calls to an AI-powered conversation:

Component	Technology	Role
SIP Trunk	SignalWire	Receives inbound calls from the public telephone network (PSTN) and converts them to SIP protocol
SIP Service	LiveKit SIP	Bridges the SIP call into a LiveKit real-time media room
LiveKit Room	LiveKit Server	A virtual room that manages audio tracks between the caller and the agent process
Agent Process	Python (livekit-agents v1.4)	Runs the voice pipeline: STT, LLM, and TTS in a single async process per call

Each component runs as a separate service. On the production server, they are orchestrated via Docker Compose at `/opt/livekit/docker-compose.yml`, all using `network_mode: host` for low-latency audio routing.

Data Flow Diagram

The following diagram shows the complete path of audio and text data during a voice call:

```
    Caller's Phone
         |
         | PSTN call (traditional phone network)
         v
  +----------------+
  |  SignalWire     |
  |  SIP Trunk     |    Converts PSTN to SIP protocol
  |  (+19458001233)|    Sends INVITE to LiveKit SIP
  +-------+--------+
          |
          | SIP INVITE (UDP/TCP port 5060)
          v
  +----------------+
  |  LiveKit SIP   |
  |  Service       |    Creates a LiveKit room for the call
  |                |    Bridges SIP audio to RTP tracks
  +-------+--------+
          |
          | RTP audio tracks (ports 10000-20000)
          v
  +----------------+
  |  LiveKit Room  |
  |  (one per call)|    Manages audio track subscriptions
  |                |    Routes audio between participants
  +-------+--------+
          |
          | Audio track subscription
          v
  +-------------------+
```

```
  |  Lumentra Agent   |
  |  (Python process) |
  |                   |
  |  +-----------+    |
  |  | Deepgram  |    |   Caller audio --> text
  |  | STT       |    |
  |  +-----+-----+    |
  |        |           |
  |        | text      |
  |        v           |
  |  +-----------+    |
  |  | OpenAI    |    |   Text --> response text
  |  | GPT-4.1   |    |
  |  | mini      |    |
  |  +-----+-----+    |
  |        |           |
  |        | text      |
  |        v           |
  |  +-----------+    |
  |  | Cartesia  |    |   Response text --> audio
  |  | Sonic-3   |    |
  |  +-----+-----+    |
  |        |           |
 +--------+-----------+
          |
          | TTS audio published to room
          v
    Caller hears response
```

Latency Breakdown

End-to-end latency (from caller finishing a sentence to hearing the agent's response) is the sum of each processing stage. The following table shows typical latencies for each hop:

Stage	Typical Latency	Notes
SIP to LiveKit Room	50-100ms	Network hop from SignalWire to LiveKit SIP service
Audio buffering	20-50ms	LiveKit collects audio frames before forwarding
STT processing	100-300ms	Deepgram Nova-3 streaming transcription; includes endpointing delay
Turn detection	50-200ms	Multilingual model + Silero VAD determines caller is done speaking
LLM first token	150-400ms	Time for GPT-4.1-mini to generate the first token of the response
LLM full response	200-800ms	Streaming; TTS begins before LLM finishes
TTS first audio chunk	80-150ms	Cartesia Sonic-3 time-to-first-byte
Audio playback start	20-50ms	LiveKit publishes audio track to room
Total (typical)	**600-1500ms**	**Perceived as natural conversation pace**

Key factors that reduce perceived latency:

- **Preemptive generation** (`preemptive_generation=True`): The LLM begins generating a response while the caller is still finishing their sentence, based on partial transcription.
- **Streaming TTS**: The TTS engine begins synthesizing audio as soon as the first LLM tokens arrive, rather than waiting for the complete response.
- **VAD prewarming**: The Silero VAD model is loaded during process startup (`prewarm` function), eliminating cold-start delay on the first call.

Connection Lifecycle

When an inbound call arrives, the system follows this sequence:

1. **SIP INVITE received**: SignalWire sends a SIP INVITE to the LiveKit SIP service, which is configured with a dispatch rule (ID: `SDR_2bZEobprwRXq`) that routes calls to the `lumentra-voice-agent` agent name.
2. **Room creation**: LiveKit SIP creates a new room for the call. The room name serves as the call identifier (used for logging, metrics, and room deletion).
3. **Agent dispatch**: The LiveKit server dispatches a job to the agent process. The `@server.rtc_session(agen voice-agent")` decorator in `agent.py` registers the entrypoint function.
4. **Room connection**: The agent calls `await ctx.connect()` to join the room, then `await ctx.wait_for_participant()` to wait for the SIP participant (the caller) to appear.
5. **SIP attribute extraction**: The agent reads the caller's phone number and the dialed business number from SIP participant attributes:
   - `sip.trunkPhoneNumber` – the business number that was dialed
   - `sip.phoneNumber` – the caller's phone number
6. **Tenant config fetch**: Using the dialed number, the agent calls `get_tenant_by_phone()` which makes an HTTP GET to `/internal/tenants/by-phone/{phone}` on the Lumentra API. This returns the system prompt, voice configuration, greeting text, escalation settings, and all tenant-specific parameters.
7. **Session start**: An `AgentSession` is created with the configured STT, LLM, TTS, VAD, and turn detection plugins. The session is started with `await session.start()`, which subscribes to the caller's audio track and begins processing.
8. **Greeting**: The `LumentraAgent.on_enter()` method fires, calling `session.generate_reply()` with the tenant's standard greeting. The caller hears the greeting within 1-2 seconds of the call connecting.
9. **Conversation loop**: The session runs an event-driven loop: caller audio flows to STT, STT output flows to the LLM, LLM output flows to TTS, and TTS audio is published back to the room. Tool calls (booking, availability checks) are executed via the Lumentra API.
10. **Call termination**: The call ends in one of three ways:
    - The `end_call` tool is invoked (customer said goodbye)
    - The duration watchdog triggers (max call duration exceeded)
    - The caller hangs up (SIP BYE signal)
11. **Shutdown callback**: The `on_shutdown` callback fires, which logs the call (transcript, duration, outcome) to the API via `/internal/calls/log`.

Process Model

The agent uses a **one-process-per-call** model with process-level isolation:

- **AgentServer**: A single long-running server process that accepts job dispatches from LiveKit.

- **JobProcess**: Each inbound call is handled in its own async context within the server. The `prewarm` function runs once per process to load the VAD model into `proc.userdata`.
- **AgentSession**: Each call gets its own `AgentSession` instance with independently configured STT, LLM, and TTS plugins. Sessions do not share state.
- **Resource cleanup**: When a call ends, the session is torn down, metrics are collected, the call is logged, and the duration watchdog task is cancelled.

Resource isolation guarantees:

- One call cannot affect another call's audio processing
- A crash in one session does not bring down the server
- Memory usage scales linearly with concurrent calls
- Each call has its own conversation history and tool context

Tenant Configuration Payload

The tenant config returned by the API contains all parameters needed to configure the agent for a specific business. Key fields:

Field	Type	Purpose
`id`	string	Tenant UUID for API calls
`system_prompt`	string	Complete system prompt built by `buildSystemPrompt()`
`business_name`	string	Used as STT keyword boost
`greeting_standard`	string	Initial greeting text
`voice_config.voice_id`	string	Cartesia voice ID for TTS
`escalation_phone`	string	Phone number for human transfers
`max_call_duration_seconds`	number	Call duration limit (default 900s)

The system prompt is assembled server-side by combining:

1. The master voice prompt (platform-level, immutable)
2. Industry-specific configuration (role, booking flow, critical rules)
3. Tenant personality settings (tone, verbosity, empathy)
4. Voice conversation guidelines
5. Business context (name, industry, location, hours, timezone)
6. Escalation configuration
7. Custom instructions from the business owner
8. Critical rules and call flow instructions

---

12. Speech-to-Text Configuration

The speech-to-text (STT) component converts the caller's spoken audio into text that the LLM can process. Lumentra uses Deepgram Nova-3, a purpose-built speech recognition model optimized for real-time voice applications.

Deepgram Nova-3 Model

Nova-3 is Deepgram's latest and most accurate speech recognition model. It is selected in the agent configuration at `agent.py` line 111:

```
stt=deepgram.STT(
    model="nova-3",
    language="multi",
    smart_format=True,
    keyterm=[tenant_config["business_name"]],
)
```

Key capabilities of Nova-3:

- **Real-time streaming**: Processes audio as it arrives, producing partial (interim) results that update as more context becomes available.
- **Multi-language support**: The `language="multi"` setting enables automatic language detection across 36+ languages without requiring the caller to select a language.
- **Low latency**: Typical transcription latency of 100-300ms from audio receipt to final transcript.
- **High accuracy**: 8-12% word error rate on conversational speech, better on clear audio.
- **Noise robustness**: Handles background noise, cross-talk, and telephone-quality audio.

Configuration Options

The STT plugin accepts several parameters that control transcription behavior:

Parameter	Value Used	Purpose
`model`	`"nova-3"`	Selects the Nova-3 speech model
`language`	`"multi"`	Enables automatic multi-language detection
`smart_format`	`True`	Applies intelligent formatting to numbers, dates, and other entities
`keyterm`	`[business_name]`	Boosts recognition accuracy for the business name

Additional Deepgram Parameters (Available) These parameters are supported by the Deepgram plugin but not currently configured in the agent. They use Deepgram's defaults:

Parameter	Default	Description
`punctuate`	True (with smart_format)	Adds punctuation to transcript
`interim_results`	`True`	Enables partial results for faster response
`utterance_end_ms`	`1000`	Milliseconds of silence before finalizing an utterance
`endpointing`	`300`	Milliseconds of silence before considering speech segment complete
`encoding`	`linear16`	Audio encoding format (matches LiveKit audio tracks)
`sample_rate`	`48000`	Audio sample rate in Hz

Multi-Language Support

The `language="multi"` setting is the recommended configuration for Lumentra because:

- Callers may speak any language without prior configuration
- The model automatically detects the language from the first few seconds of speech
- Language detection works per-utterance, so a caller can switch languages mid-call

- No tenant-level language configuration is needed

If a tenant serves a specific language market and wants to optimize accuracy, the language parameter can be set to a specific code:

Language Code	Language
`en`	English (all variants)
`es`	Spanish
`fr`	French
`hi`	Hindi
`ar`	Arabic
`zh`	Chinese (Mandarin)
`pt`	Portuguese
`multi`	Auto-detect (default)

Setting a specific language code improves accuracy by 2-5% for that language but disables recognition of other languages.

Smart Formatting

When `smart_format=True` is enabled (the current configuration), Deepgram applies intelligent formatting rules to the raw transcript:

- **Numbers**: "one hundred twenty three" becomes "123"
- **Currency**: "fifteen dollars and fifty cents" becomes "$15.50"
- **Dates**: "march second twenty twenty six" becomes "March 2nd, 2026"
- **Times**: "two thirty PM" becomes "2:30 PM"
- **Phone numbers**: "five five five one two three four" becomes "555-1234"
- **Addresses**: "one two three Main Street" becomes "123 Main Street"
- **Ordinals**: "the fifteenth" becomes "the 15th"
- **Percentages**: "twenty five percent" becomes "25%"

Smart formatting is critical for the voice agent because:

- The LLM receives properly formatted text, reducing misinterpretation
- Tool calls receive correctly formatted parameters (dates as "2026-03-15", times as "14:30")
- The agent can read back numbers and dates naturally because they are already formatted

Keyword Boosting

The `keyterm` parameter accepts a list of words or phrases that should receive boosted recognition priority:

```
keyterm=[tenant_config["business_name"]]
```

Currently, only the business name is boosted. This ensures that when a caller says the business name, Deepgram transcribes it correctly even if the name is unusual or uncommon. For example, if the business is named "Lumentra," the STT will prefer that transcription over similar-sounding words.

Keyword boosting is especially valuable for:

- Unusual business names
- Industry-specific terminology
- Proper nouns that the model might not recognize
- Menu items, service names, or product names

STT Error Handling

STT failures are handled at multiple levels:

Failure Scenario	Behavior
Deepgram service unavailable	Agent session fails to start; call is not answered
Network timeout during streaming	Transcription pauses; resumes when connection recovers
Audio quality too poor	Empty or garbled transcript; agent asks caller to repeat
Language not recognized	Falls back to closest language match; may produce lower accuracy
Keyterm not found in audio	No effect; keyterms only boost, they do not force recognition

When the STT produces no output (silence or noise), the turn detection system waits for the configured endpointing delay before the agent responds. If the caller is speaking but the STT cannot transcribe, the agent's response will be based on whatever partial text was received.

Performance Characteristics

Metric	Typical Value	Notes
Transcription latency	100-300ms	Time from audio to final text
Interim result latency	50-100ms	Time to first partial text
Word error rate (WER)	8-12%	On conversational telephone audio
Word error rate (clean)	4-8%	On clear, close-microphone audio
Supported languages	36+	With `language="multi"`
Audio format	Linear PCM, 16-bit	Streamed from LiveKit audio track
Concurrent streams	No hard limit	Limited by Deepgram plan

---

13. Text-to-Speech Configuration

The text-to-speech (TTS) component converts the agent's text responses into natural-sounding speech that the caller hears. Lumentra uses Cartesia Sonic-3, a neural TTS engine designed for real-time voice applications.

Cartesia Sonic-3 Overview

Sonic-3 is configured in the agent at `agent.py` line 118:

```
tts=cartesia.TTS(
    model="sonic-3",
    voice=tenant_config["voice_config"]["voice_id"],
    speed=0.95,
    emotion=["Content"],
)
```

Key characteristics of Sonic-3:

- **Streaming synthesis**: Begins generating audio from the first text tokens, without waiting for the complete response.
- **Low first-byte latency**: Typically 80-150ms from text input to first audio chunk.
- **Natural prosody**: Produces speech with natural rhythm, intonation, and emphasis.
- **SSML support**: Accepts Speech Synthesis Markup Language tags for pauses, speed changes, spelling, and emotion.
- **Multiple voices**: Supports a catalog of pre-built voices selectable by voice ID.
- **Consistent quality**: Maintains voice consistency across long conversations.

TTS Voice Selection

Each tenant has a voice ID configured in their voice settings (`voice_config.voice_id`). The voice ID determines which Cartesia voice the agent uses when speaking.

Voice selection is managed through the Lumentra dashboard under **Settings > Assistant > Voice**. The available voices vary by:

- **Gender**: Male and female voices available
- **Accent**: American English, British English, and other regional accents
- **Tone**: Some voices sound warm, others authoritative, others youthful

The voice ID is stored in the tenant's database record and passed to the agent as part of the tenant config payload. Changing the voice in the dashboard takes effect on the next inbound call.

Best practices for voice selection:

- Match the voice gender to the agent name (e.g., "Sarah" with a female voice)
- Choose a voice that matches the business's brand (formal business = authoritative voice, casual restaurant = warm voice)
- Test the voice by making a test call after changing it
- Avoid switching voices frequently, as repeat callers may find it disorienting

Speed Control

The speed parameter controls how fast the agent speaks. It accepts a float value:

Speed Value	Effect	Use Case
0.5	Very slow, deliberate pace	Reading back critical details like confirmation codes
0.7	Noticeably slower than normal	Elderly callers, complex information
0.9	Slightly slower than normal	Default Lumentra setting for clarity
0.95	**Current default**	Natural pace, slightly relaxed
1.0	Normal speaking speed	Standard conversational pace
1.2	Slightly faster than normal	Energetic, upbeat brand voice
1.5	Fast pace	Not recommended for phone calls
2.0	Maximum speed	Not recommended; poor intelligibility

The current default of 0.95 was chosen to:

- Sound natural without feeling rushed
- Give callers time to absorb information
- Reduce the need for callers to ask the agent to repeat itself
- Work well across accents and age groups

In addition to the base speed setting, the system prompt instructs the LLM to use `<speed ratio="0.9"/>` SSML tags when reading back important details like confirmation codes, dates, and phone numbers. This provides per-utterance speed control on top of the base speed.

Emotion Settings

The `emotion` parameter accepts a list of emotional tones that color the agent's speech. The current configuration uses `["Content"]`, which produces a warm, pleasant, and calm voice.

Available emotion values:

Emotion	Sound	When to Use
`Content`	Warm, pleasant, calm	**Default** – general conversation
`Excited`	Upbeat, energetic	Confirming a booking, good news
`Calm`	Steady, reassuring	Handling complaints, anxious callers
`Professional`	Neutral, authoritative	Business-to-business calls
`Sympathetic`	Gentle, understanding	Caller upset or frustrated
`Curious`	Engaged, interested	Asking about caller's needs
`Happy`	Bright, cheerful	Sharing positive information

The base emotion is set at the session level and applies to all speech by default. The system prompt also instructs the LLM to insert `<emotion>` SSML tags for context-specific adjustments:

- `<emotion value="Sympathetic"/>` – when the caller has a complaint
- `<emotion value="Curious"/>` – when asking about their needs
- `<emotion value="Happy"/>` – when confirming a booking

These per-response tags override the base emotion for that specific utterance only.

Streaming Architecture

TTS streaming is critical for low-latency conversation. The flow works as follows:

1. **LLM generates tokens**: The LLM streams response tokens one at a time.
2. **Text buffering**: The TTS plugin buffers tokens until it has a sentence or phrase boundary (punctuation, line break).
3. **Chunk synthesis**: Cartesia Sonic-3 synthesizes the buffered text into an audio chunk (typically 200-500ms of audio).
4. **Audio publishing**: The audio chunk is published to the LiveKit room as an audio track frame.
5. **Caller playback**: The LiveKit room routes the audio to the SIP participant, which converts it back to telephone audio.

This pipeline means the caller begins hearing the agent's response before the LLM has finished generating the full response. For a typical 2-sentence response:

Event	Time from LLM Start
First LLM token generated	0ms
First TTS audio chunk sent	80-150ms
Caller hears first word	150-250ms
LLM finishes full response	400-800ms
Caller hears last word	500-1000ms

Latency Optimization

Several techniques minimize the time between the agent deciding to speak and the caller hearing audio:

- **Streaming end-to-end**: No stage waits for the complete output of the previous stage. LLM streams to TTS, TTS streams to room.
- **Sentence-boundary buffering**: TTS starts synthesizing at the first sentence boundary rather than waiting for the full response. This reduces first-byte latency while maintaining natural prosody.
- **Audio chunk size**: Cartesia delivers audio in small chunks (50-200ms each) for smooth playback without gaps.
- **Preemptive generation**: The LLM begins generating before the caller finishes speaking, so the first token is ready sooner.
- **Connection reuse**: The TTS client maintains a persistent connection to the Cartesia API, eliminating connection setup overhead.

Fallback Behavior

When the TTS service encounters errors:

Failure Scenario	System Behavior
Cartesia API timeout	Audio generation pauses; resumes when service recovers; caller hears silence
Invalid voice ID	TTS initialization fails; agent session does not start
Rate limit exceeded	Requests are queued; caller experiences increased latency
Malformed SSML tags	Cartesia ignores invalid tags; speech is generated without the tag's effect
Network interruption	Audio stream breaks; LiveKit detects track failure; call may drop if prolonged

If the TTS service is completely unavailable, the agent session cannot start because it requires a functioning TTS to produce any output. There is no text-only fallback for voice calls.

SSML Markers Reference

The master voice prompt instructs the LLM to use these Cartesia-supported SSML markers contextually:

Marker	Syntax	Purpose
Pause	`<break time="300ms"/>`	Insert a natural pause (300ms or 500ms)
Speed	`<speed ratio="0.9"/>`	Slow down for important details
Spell	`<spell>ABC123</spell>`	Spell out letter by letter
Emotion	`<emotion value="Happy"/>`	Set emotional tone for utterance
Laughter	`[laughter]`	Natural laugh (use sparingly)

The LLM is instructed to use markers sparingly – most responses need no markers at all. Simple answers should be direct speech without any SSML.

---

14. LLM Configuration

The Large Language Model (LLM) is the brain of the voice agent – it generates the agent's responses based on the system prompt, conversation history, and tool results. Lumentra uses OpenAI GPT-4.1-mini as the primary model.

Model Selection

GPT-4.1-mini is configured in the agent at `agent.py` line 103:

```
llm = openai.LLM(
    model="gpt-4.1-mini",
    temperature=0.8,
)
```

GPT-4.1-mini was selected for the following reasons:

Criterion	GPT-4.1-mini	Alternatives Considered
Response quality	Excellent for conversation	GPT-4.1-nano too low quality for 5.6k system prompt
Tool calling	Reliable function calling	Groq free tier: 12k TPM limit, unusable
Latency	150-400ms first token	Comparable to alternatives
Cost	~$0.015 per call	Best balance of quality and cost
Streaming	Full streaming support	Required for real-time TTS
Context window	128k tokens	More than sufficient for call history

Temperature Setting

The temperature parameter controls the randomness of the LLM's responses:

Temperature	Behavior	Use Case
0.0	Deterministic, always picks most likely token	Factual lookups, data extraction
0.3	Mostly deterministic with slight variation	Customer support scripts
0.5	Balanced randomness	General conversation
0.8	**Higher variety, more natural**	**Voice conversation (current)**
1.0	Maximum meaningful randomness	Creative writing

The current setting of 0.8 was chosen because:

- Voice conversations benefit from variety – repeating the exact same phrasing sounds robotic
- Higher temperature produces more natural-sounding language with varied sentence structures
- The system prompt constrains behavior enough that higher temperature does not cause inappropriate responses
- Callers who call multiple times hear slightly different phrasing, which feels more human

Token Limits and Context Window

GPT-4.1-mini has a 128,000-token context window. For voice calls, token usage breaks down as follows:

Component	Typical Tokens	Notes
System prompt	2,000-3,000	Master voice prompt + industry config + tenant config
Conversation history	500-2,000	Grows with each turn; 10-20 turns typical
Tool call/response	100-300 per call	Availability results, booking confirmations
Agent response	20-80 per turn	Short responses by design (1-2 sentences)
Total per call	**3,000-6,000**	**Well within 128k limit**

Context window management is handled automatically by the LiveKit agents framework. The full conversation history is sent with each LLM request, which is feasible because:

- Voice calls are short (2-10 minutes typically)
- Agent responses are brief (1-2 sentences per the system prompt)
- The 128k window is never approached in normal operation

Prompt Caching

OpenAI's prompt caching significantly reduces costs for repeated calls. The cache works because:

1. The system prompt is nearly identical across calls to the same tenant (only the date changes)
2. OpenAI caches the prefix of the prompt (the system message) across requests
3. Subsequent calls to the same tenant hit the cache for the system prompt portion
4. Only the conversation turns (which are unique per call) require full processing

Observed cache performance:

Metric	Value
Cache hit rate	~88%
Cached token cost	50% of standard input price
Cache lifetime	Up to 1 hour for active tenants
Cache key	Based on system prompt prefix

This caching is automatic – no configuration is needed. The high hit rate is a natural result of the system prompt being stable across calls.

Cost Analysis

Per-call cost breakdown for a typical 3-minute call with 15 conversation turns:

Component	Cost per Call	Calculation
Deepgram STT	~$0.003	~3 min audio at $0.0059/min (Nova-3 pay-as-you-go)
OpenAI LLM (input)	~$0.004	~5k tokens at $0.40/1M (with cache discount)
OpenAI LLM (output)	~$0.003	~1.5k tokens at $1.60/1M
Cartesia TTS	~$0.005	~500 characters at $0.010/1k chars
Total per call	**~$0.015**	**At 88% prompt cache hit rate**

Cost-saving measures already in place:

- Short responses (1-2 sentences) minimize output tokens
- Prompt caching reduces input token costs by ~44%
- GPT-4.1-mini is cheaper than GPT-4.1 while maintaining conversation quality
- No unnecessary LLM calls; the agent only generates when the caller speaks

Response Streaming

The LLM streams its response token by token to the TTS engine. The streaming pipeline:

1. **Request sent**: The full conversation history (system prompt + all turns) is sent to OpenAI with `stream=True`.
2. **First token arrives**: Typically 150-400ms after the request. This is the most latency-sensitive moment.
3. **Token buffering**: Tokens accumulate in a buffer until a sentence boundary (period, question mark, exclamation) is reached.
4. **TTS handoff**: The buffered sentence is passed to Cartesia for synthesis while the LLM continues generating.
5. **Parallel processing**: The LLM generates the second sentence while the TTS synthesizes the first.
6. **Completion**: When the LLM finishes, the last buffered text is sent to TTS.

This streaming architecture means the caller hears the first sentence of the response before the LLM has finished generating the second sentence.

LLM Error Handling

The agent handles LLM errors at multiple levels:

Error Type	Detection	Response
API timeout	httpx timeout after 10s	Session retries the request; caller hears brief silence
Rate limit (429)	OpenAI returns 429 status	Automatic backoff and retry via OpenAI SDK
Model unavailable	OpenAI returns 503 status	Session fails; call cannot proceed
Invalid response	Malformed JSON or empty response	Agent generates a fallback acknowledgment
Context too long	Token limit exceeded	Should not occur given typical usage; framework handles truncation
Network failure	Connection reset or DNS failure	Agent session fails; call drops

For tool-calling errors specifically, the tools return a user-friendly fallback message ("I encountered an error. Let me try again.") rather than exposing technical details to the caller.

Model Configuration Best Practices

When adjusting LLM settings for specific use cases:

Scenario	Recommended Temperature	Notes
Medical appointment booking	0.5-0.6	Lower temperature for more predictable, precise responses
Restaurant/hospitality	0.7-0.9	Higher temperature for warmer, more varied conversation

Scenario	Recommended Temperature	Notes
Professional services	0.5-0.7	Moderate temperature for business-appropriate tone
High-volume simple tasks	0.3-0.5	Lower temperature for consistent, scripted interactions

Temperature is the only LLM parameter currently exposed at the agent level. Other parameters (max_tokens, top_p, frequency_penalty) use OpenAI defaults and are not adjusted.

---

15. Turn Detection and Endpointing

Turn detection is what makes voice conversation feel natural – it determines when the caller has finished speaking so the agent can respond, and when the caller is still thinking so the agent should wait. Getting this wrong leads to the agent either interrupting the caller or waiting too long to respond.

Overview

Lumentra uses a two-layer turn detection system:

1. **LiveKit Multilingual Turn Detection Model**: A neural model trained to detect turn boundaries in multi-language conversation.
2. **Silero VAD (Voice Activity Detection)**: A lightweight model that classifies audio frames as speech or silence.

These two systems work together – VAD detects when the caller stops making sound, and the turn detection model determines whether that silence means "I am done speaking" or "I am pausing to think."

LiveKit Multilingual Turn Detection

Configured in `agent.py` line 125:

```
turn_detection=MultilingualModel()
```

The multilingual turn detection model is a neural network trained on conversational audio in many languages. It analyzes:

- **Prosody**: Falling intonation typically signals end of turn; rising intonation signals a question or continuation.
- **Linguistic cues**: Complete sentences, trailing words, and conversation patterns.
- **Silence duration**: Combined with the above signals to make a turn-end decision.
- **Cross-language patterns**: Works across languages without per-language configuration.

The model outputs a probability score for "turn is complete" at each audio frame. The agent uses this score along with the endpointing delay configuration to decide when to begin responding.

Silero VAD (Voice Activity Detection)

Silero VAD is a lightweight neural network that classifies audio frames as speech or non-speech. It is prewarmed during process startup:

```
def prewarm(proc: JobProcess):
    proc.userdata["vad"] = silero.VAD.load()
```

And then passed to the session:

```
vad=ctx.proc.userdata["vad"]
```

VAD serves several purposes:

- **Speech detection**: Identifies when the caller is actively speaking.
- **Silence detection**: Identifies when the caller has stopped producing speech.
- **Noise rejection**: Distinguishes speech from background noise, coughs, breathing, and room sounds.
- **Efficiency**: Prevents sending non-speech audio to the STT engine, reducing API costs and processing time.

Prewarming the VAD model is important because loading the model takes 200-500ms. By loading it once during process startup, the first call handled by that process does not incur this delay.

Endpointing Configuration

Endpointing is the process of deciding that a silence gap means the caller is done speaking. The agent configures several parameters that control this behavior:

```
session = AgentSession(
    # ...
    min_endpointing_delay=0.8,
    max_endpointing_delay=2.5,
    # ...
)
```

Parameter	Value	Purpose
`min_endpointing_delay`	0.8s	Minimum silence before the agent can respond
`max_endpointing_delay`	2.5s	Maximum silence before the agent will respond regardless of turn detection

How endpointing works in practice:

1. The caller stops speaking.
2. VAD detects silence after ~100ms.
3. If the turn detection model is confident the turn is complete, the agent responds after `min_endpointing_delay` (0.8 seconds).
4. If the turn detection model is uncertain (the caller might continue), the agent waits up to `max_endpointing_delay` (2.5 seconds).
5. After 2.5 seconds of silence, the agent responds regardless of turn detection confidence.

The 0.8s minimum prevents the agent from jumping in during natural speech pauses (breathing, thinking). The 2.5s maximum prevents awkward long silences when the caller is actually done.

Interruption Handling

When the caller starts speaking while the agent is still talking, this is called a “barge-in” or interruption. The system handles this with two parameters:

```
session = AgentSession(
    # ...
    resume_false_interruption=True,
    false_interruption_timeout=1.5,
    # ...
)
```

Parameter	Value	Purpose
`resume_false_interruption`	`True`	Resume speaking if the interruption was just noise or a brief sound
`false_interruption_timeout`	1.5s	Time window to determine if an interruption was real or false

The interruption handling flow:

1. **Agent is speaking**: TTS audio is being published to the room.
2. **Caller sound detected**: VAD detects speech from the caller.
3. **Agent pauses immediately**: TTS playback stops within ~50ms of detecting caller speech.
4. **False interruption check**: The system waits up to 1.5 seconds to see if the caller actually says something meaningful.
5. **If real interruption**: The caller said something substantial. The agent discards its remaining response and processes the caller's new input.
6. **If false interruption**: The sound was a cough, "uh huh", background noise, or brief acknowledgment. The agent resumes playing the interrupted response from where it stopped.

This prevents the common problem of an agent stopping mid-sentence because the caller coughed or said "mhmm" while listening.

Preemptive Generation

```
session = AgentSession(
    # ...
    preemptive_generation=True,
    # ...
)
```

Preemptive generation allows the LLM to begin generating a response before the turn detection model has confirmed the caller is done speaking. The system:

1. Observes the partial STT transcript as the caller speaks.
2. When the caller pauses briefly, begins generating a tentative LLM response in the background.
3. If the caller resumes speaking, discards the tentative response.
4. If the turn is confirmed complete, uses the pre-generated response immediately.

This reduces perceived latency by 200-400ms because the LLM has a head start by the time the caller finishes speaking.

Configuration Summary

All turn detection and endpointing parameters from the current agent configuration:

Parameter	Value	Effect
`turn_detection`	`MultilingualModel()`	Neural turn detection model, multi-language
`vad`	`silero.VAD` (prewarmed)	Voice Activity Detection for speech/silence classification
`preemptive_generation`	`True`	LLM begins generating before turn is confirmed
`resume_false_interruption`	`True`	Resume speaking after false interruptions
`false_interruption_timeout`	`1.5s`	Time window to classify an interruption as real or false

Parameter	Value	Effect
`min_endpointing_delay`	`0.8s`	Minimum silence before agent responds
`max_endpointing_delay`	`2.5s`	Maximum silence before agent responds regardless

Common Issues and Solutions

Agent Cuts Off the Caller Mid-Sentence **Symptoms**: The agent starts responding while the caller is still talking, especially during long sentences or when the caller pauses to think.

Cause: The `min_endpointing_delay` is too short for the caller's speech pattern.

Solution: Increase `min_endpointing_delay` from 0.8s to 1.0-1.2s. This gives callers more time to pause without triggering a response. Trade-off: all responses will feel slightly slower.

Agent Waits Too Long to Respond **Symptoms**: The caller finishes speaking and there is an awkward 2-3 second silence before the agent responds.

Cause: The turn detection model is not confident the turn is complete, so it waits until `max_endpointing_delay`.

Solution: Decrease `max_endpointing_delay` from 2.5s to 1.8-2.0s. This caps the maximum wait time. Trade-off: the agent may occasionally respond before the caller finishes a multi-part thought.

Agent Stops Speaking When Caller Says "Uh Huh" **Symptoms**: The agent stops mid-sentence when the caller makes brief acknowledgment sounds like "uh huh", "mm hmm", or "yeah".

Cause: The `false_interruption_timeout` is too short, or `resume_false_interruption` is disabled.

Solution: Verify that `resume_false_interruption=True` and increase `false_interruption_timeout` to 1.5-2.0s (currently 1.5s). The system needs enough time to determine that the sound was acknowledgment, not a real interruption.

Agent Does Not Stop When Caller Interrupts **Symptoms**: The caller says "wait" or "actually" but the agent continues speaking for another second.

Cause: Audio latency in the VAD pipeline or aggressive false interruption filtering.

Solution: This is a trade-off with the false interruption handling. Decreasing `false_interruption_timeout` makes the agent more responsive to interruptions but also more likely to stop for non-interruption sounds. The current 1.5s setting is a balanced compromise.

First Call Has Higher Latency **Symptoms**: The first call after the agent process starts has noticeably higher response latency (500-1000ms extra).

Cause: The STT and TTS services need to establish connections. The VAD model should already be loaded (prewarmed), but Deepgram and Cartesia connections are established on first use.

Solution: This is expected behavior. The `prewarm` function loads the VAD model to minimize this. The STT and TTS connections are cached after the first call, so subsequent calls are faster.

Turn Detection Behaves Differently for Non-English Callers **Symptoms**: The agent interrupts more frequently or waits too long for callers speaking languages other than English.

Cause: Different languages have different turn-taking conventions. Some languages use longer pauses between thoughts; others have different intonation patterns.

Solution: The `MultilingualModel()` is trained on multiple languages and handles most patterns well. If a specific language consistently has issues, consider adjusting `min_endpointing_delay` upward (1.0-1.5s) for tenants that primarily serve that language community.

Noise Cancellation

In addition to VAD, the agent applies noise cancellation to the incoming SIP audio:

```
await session.start(
    agent=agent,
    room=ctx.room,
    room_options=room_io.RoomOptions(
        audio_input=room_io.AudioInputOptions(
            noise_cancellation=noise_cancellation.BVCTelephony(),
        ),
    ),
)
```

The `BVCTelephony` noise cancellation filter is specifically designed for telephone audio and removes:

- Background chatter and crowd noise
- Road noise and wind (callers on the go)
- Television or music in the background
- HVAC and fan noise
- Echo and feedback from speakerphone usage

This improves both STT accuracy and VAD reliability by providing cleaner audio to both systems. It runs before audio reaches the STT engine, so Deepgram receives pre-cleaned audio.

Duration Watchdog

While not strictly part of turn detection, the duration watchdog interacts with the conversation flow. Configured in `agent.py`, it enforces a maximum call duration:

Phase	Timing	Action
Normal conversation	0 to (limit - 120s)	No intervention
Wrap-up nudge	limit - 120s	Agent is instructed to start wrapping up naturally
Final warning	limit - 30s	Agent tells caller it needs to go and offers human transfer
Hard cutoff	At limit	SIP transfer to escalation phone, or room deletion

The default limit is 900 seconds (15 minutes) with a minimum of 120 seconds (2 minutes). This is configurable per tenant via `max_call_duration_seconds`.

The watchdog uses `session.generate_reply(instructions=...)` to inject instructions into the conversation without the caller hearing them. The agent incorporates these instructions naturally into its next response.

---

16. SIP Trunk Configuration

What is a SIP Trunk?

A SIP (Session Initiation Protocol) trunk is a virtual phone line that connects your business phone number to the internet. Instead of a physical phone line from a carrier, SIP trunks route calls over

IP networks. Lumentra uses SignalWire as its SIP trunk provider to receive inbound calls and bridge them into the voice agent pipeline.

SignalWire Account Setup

SignalWire provides the telephony layer. To configure a SIP trunk:

1. Create a SignalWire account at signalwire.com.
2. Purchase or port a phone number in the SignalWire dashboard.
3. Create a SIP endpoint that points to the LiveKit SIP service.
4. Note your SignalWire project ID, API token, and space URL.

The SignalWire number becomes the public-facing phone number callers dial. When a call arrives, SignalWire forwards it via SIP to the LiveKit SIP service running on the Lumentra server.

SIP Trunk Connection Details

The connection between SignalWire and LiveKit SIP uses standard SIP signaling:

Setting	Value
Protocol	SIP over UDP/TCP
Codec	PCMU (G.711 mu-law), 8kHz
SIP server	LiveKit SIP on port 5060
RTP port range	10000-20000
WebRTC ICE range	50000-60000
Authentication	IP-based allowlisting

The LiveKit SIP service (`livekit-sip`) listens on port 5060 for incoming SIP INVITE messages. When it receives a call, it creates a LiveKit room and bridges the SIP audio into that room as a participant.

Call Routing with Dispatch Rules

Dispatch rules determine which agent handles an incoming call. LiveKit uses dispatch rules to map SIP trunk phone numbers to named agents:

Component	Purpose
SIP Trunk	Receives call from SignalWire
Dispatch Rule	Maps trunk to agent name
Agent Name	Identifies which code runs

In Lumentra, the dispatch rule routes all calls on the configured trunk to the agent named `lumentra-voice-agent`. The agent code in `agent.py` registers itself with this name:

```
@server.rtc_session(agent_name="lumentra-voice-agent")
```

Each inbound call creates a new LiveKit room. The agent joins the room, fetches tenant configuration based on the dialed number, and begins the conversation.

Caller ID Extraction

When a call arrives via SIP, the caller's phone number and the dialed number are embedded in the SIP headers. The LiveKit SIP service extracts these and attaches them as participant attributes:

Attribute	SIP Header Source
`sip.phoneNumber`	From (caller number)
`sip.trunkPhoneNumber`	To (dialed number)

The agent reads these attributes after the SIP participant joins:

```
dialed_number = participant.attributes.get(
    "sip.trunkPhoneNumber", ""
)
caller_phone = participant.attributes.get(
    "sip.phoneNumber", ""
)
```

The dialed number identifies which tenant (business) the call is for. The caller phone number is used for contact lookup, booking records, and call logging.

Number Porting

Businesses can bring their existing phone number to Lumentra:

1. Submit a port request through the dashboard (**Settings > Phone > Port**).
2. Provide your current carrier details and account number.
3. SignalWire processes the port (typically 5-15 business days).
4. During porting, the number continues working on the old carrier.
5. Once ported, calls route directly through SignalWire to Lumentra.

Alternatively, businesses can use call forwarding from their existing carrier to a Lumentra-provisioned number while keeping their original number active.

Concurrent Call Limits

Each LiveKit room handles one call. The system can handle multiple concurrent calls by creating separate rooms:

Factor	Limit
LiveKit rooms per server	Hundreds (CPU-bound)
SIP concurrent channels	Per SignalWire plan
Agent instances	Auto-scaled by LiveKit
RTP port range	10000 ports available

The LiveKit agent framework automatically spawns worker processes for concurrent calls. The `prewarm` function pre-loads the VAD model so that new calls start with lower latency:

```
def prewarm(proc: JobProcess):
    proc.userdata["vad"] = silero.VAD.load()
```

---

17. System Prompt Engineering

How the System Prompt is Built

Every call begins with the agent receiving a system prompt – a large block of instructions that tells the LLM who it is, how to behave, and what it knows about the business. This prompt is assembled dynamically by the `buildSystemPrompt` function in `chat.ts` each time a tenant configuration is fetched.

The prompt is constructed by concatenating layers in a specific order. Each layer serves a distinct purpose and cannot be overridden by layers below it.

Prompt Assembly Order

The system prompt is assembled in this sequence:

Order	Layer	Source
1	Master Voice Prompt	`master-voice-prompt.ts`
2	Agent Identity	Tenant config
3	Personality Traits	Tenant config
4	Voice Guidelines	Hardcoded
5	Business Context	Tenant config
6	Operating Hours	Tenant config
7	Escalation Info	Tenant config
8	Industry Rules	`industry-prompts.ts`
9	Custom Instructions	Tenant config
10	Final Critical Rules	Hardcoded
11	Call Flow	Hardcoded

Layers 1, 4, 10, and 11 are immutable platform-level instructions. Business owners can only modify layers 2, 3, 5, 6, 7, and 9 through the dashboard. Layer 8 is determined by the tenant's industry selection.

Variables Injected into the Prompt

The following dynamic values are injected at build time:

Variable	Source	Example
`agentName`	`tenant.agent_name`	"Sarah"
`businessName`	`tenant.business_name`	"Grand Plaza Hotel"
`industry`	`tenant.industry`	"hotel"
`personality.tone`	`tenant.agent_personality`	"friendly"
`personality.verbosity`	`tenant.agent_personality`	"concise"
`personality.empathy`	`tenant.agent_personality`	"high"
`locationAddress`	`tenant.location_address`	"123 Main St"
`locationCity`	`tenant.location_city`	"Austin, TX"
`timezone`	`tenant.timezone`	"America/Chicago"
`operatingHours`	`tenant.operating_hours`	Schedule array
`escalationPhone`	`tenant.escalation_phone`	"+15551234567"
`customInstructions`	`tenant.custom_instructions`	Free-form text
Today's date	`new Date()`	"2026-03-02"

Persona Definition

The agent's persona is constructed from the `agentName` and `businessName` variables combined with the industry role description:

```
You are {agentName}, the receptionist at {businessName}.
```

The industry config adds a role description (e.g., "You help guests check room availability, make reservations, and answer questions.") followed by a universal reinforcement: "You ARE the front desk - speak as a real person, not an assistant or AI."

This persona framing is critical. If the caller asks "Are you a robot?" the agent is instructed to deflect: "I'm here to help with your reservation" – never admitting to being automated.

Personality Configuration

The personality section is built from three independent axes:

Tone options:

Value	Prompt Instruction
`professional`	Maintain professional demeanor
`friendly`	Warm and approachable
`casual`	Relaxed, like a friend
`formal`	Formal language and etiquette

Verbosity options:

Value	Prompt Instruction
`concise`	Brief, 1-2 sentences max
`balanced`	Enough detail to be helpful
`detailed`	Thorough explanations

Empathy options:

Value	Prompt Instruction
`high`	Acknowledge emotions, validate
`medium`	Understanding, situational
`low`	Focus on efficiency

Instruction Ordering

The order in which instructions appear in the system prompt matters for LLM behavior. Instructions placed earlier carry more weight:

1. **Master Voice Prompt** (first): Establishes HOW the agent speaks. Cannot be overridden.
2. **Identity and Personality** (middle): Defines WHO the agent is.
3. **Business Context** (middle): Provides WHAT the agent knows.
4. **Industry and Custom Rules** (middle-late): Adds domain-specific behavior.
5. **Final Critical Rules** (last): Reinforces the most important constraints.

The “sandwich” pattern (critical rules at top AND bottom) ensures the LLM respects core constraints even if custom instructions in the middle are verbose or contradictory.

Industry-Specific Prompt Templates

Each supported industry provides four sections:

Hotel: - Role: Room availability, reservations, guest questions. - Rules: State tracking (check-in, nights, guests, room, name), confusion handling, strict transfer rules. - Flow: Flexible 7-step process adapting to info already given. - FAQ: Check-in/out, parking, WiFi, cancellation, pets.

Restaurant: - Role: Table reservations, menu, orders, host behavior. - Rules: State tracking (date, time, party, name, occasion), 15-minute hold policy, large party handling. - Flow: Natural 9-step process with name spelling. - FAQ: Hours, dress code, parking, events, takeout.

Medical: - Role: Appointments, prescription messaging, practice info. - Rules: No medical advice. Emergency to 911. HIPAA language. Patient ID verification (name + DOB). - Flow: 6-step distinguishing new vs. existing patients. - FAQ: Insurance, hours, new patient forms, cancellation.

Dental: - Role: Dental appointments, services, office info. - Rules: No dental advice. Emergency handling. No cost quotes without insurance. - Flow: 5-step distinguishing routine vs. concern.

Salon: - Role: Service appointments, pricing, salon info. - Rules: Ask service type and stylist preference. - Flow: 6-step with stylist preference. - FAQ: Walk-ins, cancellation, products.

Auto Service: - Role: Vehicle service, questions, shop info. - Rules: Collect vehicle year/make/model, symptoms. - Flow: 7-step with vehicle details. - FAQ: Estimates, warranty, wait time.

Token Budget Considerations

The system prompt consumes tokens on every LLM call:

Impact	Details
Higher latency	More tokens to process
Higher cost	Prompt tokens billed per call
Cache efficiency	OpenAI caches repeated prefixes

Approximate prompt sizes by industry:

Industry	Token Count (approx.)
Hotel	5,000-5,600
Restaurant	5,000-5,500
Medical	5,200-5,800
Dental	4,800-5,200
Salon	4,500-5,000
Auto Service	4,600-5,000
Generic	4,500-5,000

With OpenAI prompt caching at approximately 88% hit rate, the effective cost per call is around $0.015.

Best Practices for Custom Instructions

Custom instructions appear under “Business-Specific Instructions” near the end of the prompt.

Do:

- List specific services and prices.
- Describe staff by name and specialty.
- Include policies (cancellation, deposits).
- Mention seasonal specials or promotions.
- Specify what the agent should NOT discuss.
- Keep instructions under 500 words.

Do not:

- Contradict the master voice prompt.
- Write extremely long text (adds cost).
- Add formatting or speech instructions.
- Reference internal systems or databases.
- Write in non-English unless the tenant primarily serves non-English speakers.

Example for a dental office:

```
Dentists: Dr. Sarah Chen (general, cosmetic),
Dr. James Park (orthodontics, oral surgery).
```

```
Services and times:
- Cleaning: 45 min, $150 without insurance
- Whitening: 1 hour, $300
- Crown: 2 appointments, $800-1200

48-hour cancellation notice required.
$50 late cancellation fee.
New patients get free consultation.
We accept Delta Dental, Cigna, Aetna.
```

---

18. Tool Execution Deep Dive

Available Tools

The voice agent has six tools that perform real actions during a call. Each is a Python function decorated with `@function_tool()` in `tools.py`:

Tool	Purpose
`check_availability`	Query open time slots
`create_booking`	Create a confirmed booking
`create_order`	Place a food order
`transfer_to_human`	Transfer call to staff
`end_call`	Terminate the call
`log_note`	Save a note about the caller

How Tools are Registered

Tools are registered in the `LumentraAgent` constructor:

```python
class LumentraAgent(Agent):
    def __init__(self, tenant_config, caller_phone, job_ctx):
        super().__init__(
            instructions=tenant_config["system_prompt"],
            tools=[
                check_availability,
                create_booking,
                create_order,
                transfer_to_human,
                end_call,
            ],
        )
```

The `@function_tool()` decorator converts each function into a schema sent to the LLM. The LLM sees the function name, its docstring as description, and parameter types.

Tool Call Flow

```
LLM generates tool call (name + arguments)
        |
        v
LiveKit framework intercepts the call
        |
        v
Python @function_tool handler executes
        |
```

```
        v
_call_tool() sends HTTP POST to API
        |
        v
POST /internal/voice-tools/:action
        |
        v
API executeTool() routes to handler
        |
        v
Tool runs (DB queries, business logic)
        |
        v
Result returned to agent as string
        |
        v
LLM generates natural language for caller
```

The _call_tool Helper

All tools route through a common helper:

```python
async def _call_tool(context, action, args):
    resp = await client.post(
        f"/internal/voice-tools/{action}",
        json={
            "tenant_id": ...,
            "call_sid": ...,
            "caller_phone": ...,
            "escalation_phone": ...,
            "args": args,
        },
    )
```

This handles authentication (Bearer token), error catching, and response parsing. Returns a string message for the LLM.

check_availability

Queries open appointment slots for a date.

Parameters:

Parameter	Type	Required	Description
`date`	string	Yes	YYYY-MM-DD format
`service_type`	string	No	Service type requested

Logic:

1. Queries `bookings` for non-cancelled entries on the date.
2. Compares against hourly slots 09:00-17:00.
3. Returns up to 3 available slots formatted for voice.
4. If none available, suggests another date.

Response:

Field	Type	Description
`available`	boolean	Whether slots exist
`slots`	string[]	Available time slots
`message`	string	Voice-friendly message

create_booking

Creates a confirmed booking in the database.

Parameters:

Parameter	Type	Required	Description
`customer_name`	string	Yes	Spelled and confirmed
`customer_phone`	string	Yes	Phone number
`date`	string	Yes	YYYY-MM-DD format
`time`	string	Yes	HH:MM (24-hour)
`service_type`	string	No	Defaults to “general”
`notes`	string	No	Special requests

Logic:

1. Generates 6-char code from `ABCDEFGHJKLMNPQRSTUVWXYZ23456789` (no ambiguous chars).
2. Links to current call record if found.
3. Inserts into `bookings` with status “confirmed”.
4. Returns code and voice-formatted date/time.

Response:

Field	Type	Description
`success`	boolean	Whether created
`booking_id`	string	Database ID
`confirmation_code`	string	6-char code
`message`	string	Confirmation message

create_order

Places a food order (pickup or delivery).

Parameters:

Parameter	Type	Required	Description
`customer_name`	string	Yes	Name for the order
`order_type`	string	Yes	“pickup” or “delivery”
`items`	string	Yes	Comma-separated items
`customer_phone`	string	No	Falls back to caller ID
`delivery_address`	string	Cond.	Required for delivery
`special_instructions`	string	No	Special requests

Validation:

Rejected Value	Response
Empty name	“I need a name for the order.”
Invalid order type	“Pickup or delivery?”
Empty items	“What would you like?”
Delivery, no address	“Delivery address needed.”

Estimated times: Pickup 20 min, delivery 40 min.

transfer_to_human

Transfers the caller to a human staff member.

Parameters:

Parameter	Type	Required	Description
`reason`	string	Yes	customer_request, complaint, refund_request, cannot_resolve

Logic:

1. API updates call outcome to “escalation”.
2. Finds SIP participant in LiveKit room.
3. SIP REFER transfer to escalation phone.
4. If no phone configured, tells caller someone will call back.

SIP transfer:

```
sip_uri = f"sip:{phone}@sip.signalwire.com"
await sip_participant.transfer_sip_call(sip_uri)
```

end_call

Terminates the call after conversation is complete.

Parameters:

Parameter	Type	Required	Description
`reason`	string	Yes	conversation_complete, customer_requested_hangup, order_confirmed, booking_confirmed

Logic:

1. Logs end reason to API (best-effort).
2. Waits 1 second for final TTS audio.
3. Deletes LiveKit room (immediate disconnect).
4. Falls back to `session.shutdown()`.

log_note

Saves a note about the caller to the contact record.

Parameters:

Parameter	Type	Required	Description
`note`	string	Yes	Note content
`note_type`	string	No	general, preference, complaint, compliment, follow_up, internal

Logic:

1. Finds or creates contact by caller phone.
2. Links note to current call record.
3. Inserts into `contact_notes` table.

Internal API Endpoints

Endpoint	Method	Purpose
`/internal/voice-tools/:action`	POST	Execute tool action
`/internal/calls/log`	POST	Log completed call

Both require Bearer token matching `INTERNAL_API_KEY`. The Python agent uses a shared `httpx.AsyncClient`:

```
_client = httpx.AsyncClient(
    base_url=API_URL,
    headers={"Authorization": f"Bearer {API_KEY}"},
    timeout=10.0,
)
```

Rate Limiting on Tool Calls

Tool calls are rate-limited by the LLM's own behavior – the system prompt instructs the agent to collect all required information before calling a tool, preventing rapid-fire calls. Additionally:

- The httpx client has a 10-second timeout per request.
- The API processes tool calls sequentially per call (one room = one agent = one tool at a time).
- OpenAI's rate limits on function calling apply (typically generous for gpt-4.1-mini).

Error Handling in Tool Calls

Level	Handler	Behavior
Python tool handler	try/except	Returns generic error message
API endpoint	try/catch	Returns HTTP 500 with logs
LLM behavior	System prompt	"Let me try that again"

Tool results are truncated to 200 characters in agent logs.

---

19. Call Logging and Analytics

How Calls Are Logged

Call logging is handled by `call_logger.py`. When a call ends, the `on_shutdown` callback invokes `log_call`:

```
await asyncio.wait_for(
    log_call(
        tenant_id=tenant_config["id"],
        call_sid=ctx.room.name,
        caller_phone=caller_phone,
        session=session,
        started_at=call_started_at,
    ),
    timeout=5.0,
)
```

The 5-second timeout prevents blocking session teardown.

Data Captured Per Call

Field	Source	Description
`tenant_id`	Tenant config	Business identifier
`call_sid`	Room name	Unique call identifier
`caller_phone`	SIP attributes	Caller phone number
`direction`	Always "inbound"	Call direction
`status`	Mapped from agent	completed/failed/missed
`started_at`	Agent start time	UTC ISO timestamp
`ended_at`	Current time	UTC ISO timestamp
`duration_seconds`	Calculated	End minus start
`outcome_type`	Transcript analysis	booking/inquiry/escalation
`outcome_success`	Derived	True for booking/inquiry
`transcript`	Session history	Full conversation text
`summary`	Generated	Turns, duration, topics

Transcript Extraction

Built from the session's chat history:

```
for msg in session.history.messages():
    if msg.role in ("user", "assistant"):
        speaker = "Customer" if role == "user" else "Agent"
        lines.append(f"{speaker}: {text}")
```

System messages and tool calls are excluded.

Outcome Detection

The `_detect_outcome` function uses regex on the transcript:

Outcome	Pattern Matched
`booking`	confirmation code, booked, confirmed, reserved
`booking`	order placed, confirmed, delivery, pickup
`escalation`	transfer, human, manager, speak to
`inquiry`	Default (no match)

Summary Generation

The `_build_summary` function:

1. Counts conversation turns.
2. Detects topics by keyword scanning.
3. Formats: "{N} turns, {D}s. Topics: {list}."

Topic	Keywords
booking	book, appoint, reserv, schedul
availability	avail, open, slot, time
ordering	order, deliver, pickup, menu
business info	hour, open, close, locat, address
escalation	transfer, human, manager, complain
general inquiry	Default

API Call Logging Endpoint

`/internal/calls/log` POST performs:

1. Validates `tenant_id` and `call_sid`.
2. Finds or creates contact by caller phone.
3. Inserts into `calls` table.
4. Triggers post-call automation.
5. Returns new call record ID.

Status mapping:

Agent Status	DB Status
completed	completed
failed	failed
transferred	completed
no-answer	missed
(other)	completed

Post-Call Processing

`runPostCallAutomation` runs asynchronously:

Call Outcome	Automation
Booking	Create/update deal in CRM
Escalation	Create follow-up task
Inquiry	Update contact record
All calls	Link contact to call

Sentiment Analysis

The logging endpoint accepts an optional `sentiment_score` field. Currently not auto-calculated – reserved for future implementation.

Dashboard Call Analytics

The **Calls** page provides:

View	What It Shows
Call list	All calls with outcome badges
Detail panel	Metadata, transcript, contact
Search	Filter by name or phone
Outcome filter	Filter by outcome type
Export	Download call data

Detail panel contents:

- Caller name and phone number
- Duration, date/time, outcome, direction
- Auto-generated summary
- Detected intents as badges
- Linked contact card
- Linked booking (if created)
- Audio recording (if enabled)
- Full labeled transcript

Call Metrics

Metric	Description
Booking conversion rate	Calls resulting in a booking
Average call duration	Typical call length
Escalation rate	Calls transferred to humans
Hangup rate	Calls ending without resolution
Repeat caller rate	Same number calling again

Pipeline Metrics (UsageCollector)

Metric	Description
LLM prompt tokens	Input tokens per LLM call
LLM completion tokens	Output tokens per LLM call
STT processing time	Deepgram latency
TTS generation time	Cartesia latency
Total call duration	Wall-clock length

---

20. Error Handling and Fallbacks

STT Failure Handling

Deepgram Nova-3 processes audio via WebSocket:

Failure Mode	Behavior
Connection lost	LiveKit auto-reconnects
Transcription timeout	No text; caller hears silence
Garbled transcript	Agent: “Sorry, what was that?”
Service unavailable	Session cannot start

LLM Failure Handling

GPT-4.1-mini failures:

Failure Mode	Behavior
API timeout	httpx 10s timeout; fails
Rate limiting (429)	SDK retries with backoff
Invalid response	Fallback acknowledgment
Context overflow	Oldest turns dropped
Service outage	Cannot generate responses

TTS Failure Handling

Cartesia Sonic-3 failures:

Failure Mode	Behavior
Connection lost	No audio; silence
Generation timeout	Response lost for turn
Invalid SSML	Markers ignored; text plays
Service unavailable	Agent cannot speak

Network Dependencies

Dependency	Failure Impact
SignalWire (SIP)	No new calls connect
Deepgram (STT)	Agent cannot hear caller
OpenAI (LLM)	Agent cannot respond
Cartesia (TTS)	Agent cannot speak
Lumentra API	Tools and logging fail

Internal API uses local address (`http://10.0.1.5:3100`), isolated from external network issues.

API Error Handling

Tool calls fail gracefully:

```
except Exception as e:
    logger.error("Tool %s error: %s", action, e)
    return "I encountered an error. Let me try again."
```

The caller never sees technical error details.

Graceful Call Termination

On fatal errors:

1. Agent attempts brief goodbye.
2. 1-second grace for final TTS audio.
3. LiveKit room deleted (immediate disconnect).
4. Fallback to `session.shutdown()`.
5. `on_shutdown` fires to attempt logging.

Call logger timeout:

```python
await asyncio.wait_for(log_call(...), timeout=5.0)
```

Duration Watchdog

Phase	Timing	Action
Phase 1	limit - 120s	Nudge to wrap up
Phase 2	limit - 30s	Offer human transfer
Phase 3	limit reached	Force transfer or end

Default: 900s (15 min). Minimum: 120s (2 min). Watchdog cancelled on normal call end:

```python
except asyncio.CancelledError:
    logger.debug("Watchdog cancelled")
```

Error Logging Levels

Python agent:

Level	Usage
DEBUG	Watchdog cancel, routine events
INFO	Tool calls, call lifecycle, metrics
WARNING	Missing tenant, contact fail, timeout
ERROR	Tool fail, transfer fail, API error

API server:

Prefix	Usage
`[INTERNAL]`	Internal API route logs
`[TOOLS]`	Tool execution logs

Recovery Strategies

Scenario	Recovery
Tool call fails	Agent: "Let me try again"
API unreachable	Tools return error; agent adapts
STT drops briefly	Agent waits; STT reconnects
TTS drops briefly	Silence; TTS reconnects
Caller disconnects	on_shutdown fires; call logged
Agent process crashes	LiveKit detects; room cleaned

---

21. Multi-Language Support

Supported Languages

Lumentra uses Deepgram Nova-3 with `language="multi"` for automatic multilingual transcription:

Language	Code
English	en
Spanish	es
French	fr
German	de
Italian	it
Portuguese	pt
Dutch	nl
Hindi	hi
Japanese	ja
Korean	ko
Chinese (Mandarin)	zh
Russian	ru
Turkish	tr
Arabic	ar
Polish	pl
Ukrainian	uk
Swedish	sv
Norwegian	no
Danish	da
Finnish	fi

Not exhaustive. Deepgram adds languages continually.

Per-Tenant Language Configuration

All tenants use multilingual mode:

```
stt=deepgram.STT(
    model="nova-3",
    language="multi",
    smart_format=True,
    keyterm=[tenant_config["business_name"]],
)
```

`keyterm` boosts business name recognition. `smart_format` applies language-appropriate formatting. Future versions may allow per-tenant language restriction.

Language Detection

Detection happens automatically at the STT layer:

Behavior	Details
Detection speed	First few seconds of speech
Per-utterance	Language can change mid-call
No selection needed	Automatic from audio
Confidence-based	Falls back to closest match

Multilingual System Prompts

The system prompt is always English. For non-English callers:

1. Deepgram transcribes into detected language text.
2. LLM receives English instructions + non-English input.
3. LLM responds in caller's language.

4. Cartesia renders response in that language.

GPT-4.1-mini follows English instructions while generating non-English output. Industry rules are English-only, which may cause subtle adherence issues.

TTS Voice Selection Per Language

Cartesia Sonic-3 supports multilingual synthesis:

```
tts=cartesia.TTS(
    model="sonic-3",
    voice=tenant_config["voice_config"]["voice_id"],
    speed=0.95,
    emotion=["Content"],
)
```

Sonic-3 renders the same voice ID in multiple languages with automatic pronunciation adjustment. Some voices sound more natural in certain languages.

Turn Detection for Multiple Languages

The multilingual model handles varied speech patterns:

```
turn_detection=MultilingualModel()
```

Parameters:

Parameter	Value	Purpose
`min_endpointing_delay`	0.8s	Min silence before response
`max_endpointing_delay`	2.5s	Max wait for speech
`false_interruption_timeout`	1.5s	Grace for brief sounds
`resume_false_interruption`	True	Resume after false interrupts
`preemptive_generation`	True	Generate during pauses

Limitations and Known Issues

Limitation	Details
English-only system prompt	Non-English adherence may degrade
English transcript labels	“Customer”/“Agent” always English
English outcome detection	Keyword matching English only
English summary generation	Topic keywords English only
Code-switching	Mid-sentence language change errors
Heavy accents	Reduced STT accuracy
Non-English custom instructions	May not integrate cleanly
SSML markers	Designed for English patterns

For non-English-focused tenants, writing custom instructions in the primary caller language improves adherence.

---

22. Operating Hours and After-Hours Behavior

Business Hours Configuration

Operating hours are configured per tenant in **Settings > Hours** as JSON in the `operating_hours` column:

```json
{
  "schedule": [
    { "day": "Monday", "open": "09:00", "close": "17:00" },
    { "day": "Tuesday", "open": "09:00", "close": "17:00" },
    { "day": "Wednesday", "open": "09:00", "close": "17:00" },
    { "day": "Thursday", "open": "09:00", "close": "17:00" },
    { "day": "Friday", "open": "09:00", "close": "20:00" },
    { "day": "Saturday", "open": "10:00", "close": "15:00" }
  ],
  "holidays": ["2026-12-25", "2026-01-01"]
}
```

Days not listed are treated as closed.

How Hours are Injected into the Prompt

`buildSystemPrompt` formats the schedule:

```
## Operating Hours
- Monday: 09:00 - 17:00
- Tuesday: 09:00 - 17:00
- Wednesday: 09:00 - 17:00
- Thursday: 09:00 - 17:00
- Friday: 09:00 - 20:00
- Saturday: 10:00 - 15:00
- Closed on: 2026-12-25, 2026-01-01
```

Followed by: “Use these hours to answer ‘when are you open?’ questions.”

The code that generates this:

```typescript
if (options?.operatingHours?.schedule?.length > 0) {
    prompt += "\n## Operating Hours\n";
    for (const slot of schedule) {
        prompt += `- ${slot.day}: ${open} - ${close}\n`;
    }
    if (holidays?.length > 0) {
        prompt += `- Closed on: ${holidays.join(", ")}\n`;
    }
}
```

How the Agent Checks Hours

The agent uses the schedule in its prompt (not programmatic checks):

Caller Question	Agent Behavior
“When are you open?”	Reads schedule from prompt
“Open on Sunday?”	Sunday not listed = closed
“Book at 7 PM Monday?”	Compares to Mon close (17:00)
“Open on Christmas?”	Checks holidays list
“What are your hours?”	Lists relevant days

After-Hours Greeting

Three greeting types per tenant:

Greeting	Config Field	When Used
Standard	`greeting_standard`	Business hours
After-hours	`greeting_after_hours`	Outside hours
Returning	`greeting_returning`	Repeat callers

The agent greets via `on_enter`:

```
async def on_enter(self):
    self.session.generate_reply(
        instructions=f"Greet the caller: {self.tenant_config['greeting_standard']}"
    )
```

After-hours detection is handled by the LLM using today's date and the schedule in the system prompt.

After-Hours Behavior Options

Behavior	Description
Take a message	Collect name, phone, reason
Offer callback	Inform when business reopens
Book next open day	Offer next-day appointment
Emergency redirect	Provide emergency number
Limited service	Answer basic questions only

Industry-specific defaults:

Industry	After-Hours Behavior
Medical	"If emergency, call 911." Take msg.
Dental	Emergency appointment if available.
Hotel	Full booking available 24/7.
Restaurant	Inform next opening. Take reservation.
Salon	Take message. Mention walk-ins.
Auto Service	Take message with vehicle details.

Voicemail Handling

When the agent determines the business is closed or cannot help, the voicemail system can be triggered. The voicemail service (`voicemail.ts`) supports:

Trigger Reason	Description
`after_hours`	Business is closed
Other triggers	Agent cannot resolve request

Voicemail records are stored in the `voicemails` table with:

Field	Description
`tenant_id`	Business identifier
`call_sid`	Call identifier
`caller_phone`	Caller's number
`recording_url`	Audio recording URL

Field	Description
`duration_seconds`	Voicemail length
`transcript`	Transcribed voicemail text
`reason`	Why voicemail was triggered
`status`	pending, reviewed, resolved

Holiday Schedule Configuration

Holidays are ISO date strings in `operating_hours.holidays`:

1. Navigate to **Settings > Hours**.
2. Add dates to the holidays list.
3. Save configuration.

Common holidays:

Holiday	Typical Date
New Year's Day	2026-01-01
Memorial Day	2026-05-25
Independence Day	2026-07-04
Labor Day	2026-09-07
Thanksgiving	2026-11-26
Christmas Day	2026-12-25

Timezone Handling

Each tenant has a `timezone` field (IANA format):

```
Timezone: America/Chicago
```

Purpose	How It Helps
Agent awareness	Knows business timezone
Date context	Current day + TZ = open/closed

Time-sensitive scenarios:

Scenario	How Timezone Helps
"Open right now?"	Checks day against schedule
"What time close today?"	Reads today's close time
"Can I come at 4?"	Compares to today's hours
Caller in different TZ	Responds in business timezone

Configuration Summary

Setting	Location	Format
Weekly schedule	Settings > Hours	Array of day/open/close
Holiday dates	Settings > Hours	Array of YYYY-MM-DD
Timezone	Settings > Hours	IANA timezone string
Standard greeting	Settings > Greetings	Free-form text
After-hours greeting	Settings > Greetings	Free-form text
Returning greeting	Settings > Greetings	Free-form text

Setting	Location	Format
After-hours behavior	Custom Instructions	Free-form text

---

23. Call Transfer and Escalation

When the Agent Transfers Calls

The voice agent transfers calls to human staff based on escalation rules configured per tenant. Transfer logic lives in two places: the `transfer_to_human` tool in `tools.py` and the duration watchdog in `agent.py`. The agent does not decide on its own when to transfer – it follows the rules defined in the tenant configuration and the system prompt.

Escalation is triggered by one of these conditions:

Trigger	Source	Example
Caller requests human	System prompt rule	"Let me talk to a person"
Caller demands manager	System prompt rule	"I want the manager"
Complaint or refund	System prompt rule	"This is unacceptable"
Agent cannot resolve	Tool: transfer_to_human	Repeated failed lookups
Call duration exceeded	Duration watchdog	Call hits max_call_duration

The system prompt explicitly instructs the agent to only transfer when the caller uses specific phrases like "human", "real person", "manager", or "I want to complain". General frustration or confusion does not trigger transfer – the agent is instructed to stay calm and continue helping.

Transfer Types

Blind Transfer (SIP REFER)

The primary transfer method uses SIP REFER via LiveKit's `transfer_sip_call` method. The agent constructs a SIP URI from the escalation phone number:

```
sip:{escalation_phone}@sip.signalwire.com
```

The caller is transferred directly to the destination without the agent bridging the call. Once the REFER is sent, the agent's involvement ends. This is a blind transfer – the destination receives the call with no context from the agent beyond what the caller communicates.

Message-Based Fallback

When no escalation phone is configured or the SIP transfer fails, the agent takes a message instead. The agent tells the caller "Someone will call you back shortly" and logs the escalation reason through the API. This creates a follow-up task in the CRM pipeline for the escalation contact.

Transfer Destinations

Transfer destinations are configured per tenant in the escalation settings:

Setting	Description
escalation_phone	Primary transfer phone number
escalation_enabled	Toggle to enable/disable
escalation_triggers	List of trigger conditions

The escalation phone is a single phone number. The system does not currently support multiple departments or routing to different numbers based on the reason for transfer. All transfers go to the same destination.

Escalation Triggers

The `escalation_triggers` field stores an array of conditions. When the agent detects one of these conditions during conversation, it invokes the `transfer_to_human` tool with the matching reason code:

Reason Code	Description
customer_request	Caller asked for a human
complaint	Caller is making a complaint
refund_request	Caller wants a refund
cannot_resolve	Agent tried but cannot help

The tool posts to `/internal/voice-tools/transfer_to_human` on the API, which logs the escalation event and returns a response message for the agent to speak before transferring.

Post-Transfer Logging

When a transfer occurs, the system logs it in two ways:

1. **Tool call log**: The `transfer_to_human` tool posts the reason to the API, which records the escalation in the call record.
2. **Call completion log**: When the session ends after transfer, the `on_shutdown` callback fires `log_call`, which detects "transfer" or "human" keywords in the transcript and classifies the outcome as "escalation".

Post-call automation then creates a follow-up task for the escalation contact in the CRM.

Fallback When Transfer Target is Unavailable

If the SIP REFER transfer fails (network error, invalid number, destination not answering), the agent catches the exception and falls back to message-taking:

```
"I was unable to transfer you directly.
 Someone will call you back shortly."
```

The call continues with the agent rather than dropping. The escalation is still logged so staff can follow up. If the duration watchdog triggers the transfer and it fails, the fallback is to delete the LiveKit room (end the call), since the call has already exceeded its time limit.

---

24. Greeting and Farewell Customization

Default Greeting Templates

Each tenant has three greeting slots stored in the database:

Greeting Type	Field Name	When Used
Standard	greeting_standard	Default for all calls
After-hours	greeting_after_hours	Outside operating hours
Returning caller	greeting_returning	Recognized phone number

The agent delivers the greeting through the `on_enter` method in `LumentraAgent`, which calls `session.generate_reply` with the greeting text as an instruction. The LLM uses the greeting as a starting point but may adapt it slightly based on its personality settings.

Custom Greeting Configuration

Greetings are configured in the dashboard under Settings > Greetings. The greeting text should be written as natural speech, not a script. Examples by industry:

Hotel: "Thanks for calling Grand Plaza Hotel, this is Sarah. How can I help you?"

Medical Office: "Thank you for calling Westside Family Medicine. How can I help you today?"

Restaurant: "Thanks for calling Bella Cucina, how can I help?"

Salon: "Hey there, thanks for calling Luxe Salon. What can I do for you?"

Guidelines for effective greetings:

- Keep it under 20 words
- Include the business name
- Include the agent name for a personal touch
- End with an open question
- Match the tone to your personality setting
- Avoid "Good morning/afternoon" unless your agent should use time-aware greetings

Dynamic Greeting Elements

The system prompt includes today's date, which the LLM can use to adapt greetings contextually. The agent also has access to the caller's phone number via SIP attributes, but caller name lookup is not currently implemented in the greeting flow.

Time-of-day awareness depends on the timezone configured for the tenant. The LLM receives the timezone in the system prompt and can use it to say "Good morning" or "Good evening" if the greeting text suggests it.

Farewell Messages and Call Closing

Call ending is governed by rules in the master voice prompt. The agent follows this flow:

1. Caller's request is fully handled
2. Caller says goodbye or "that's all"
3. Agent gives a brief farewell: "You're all set. Bye!"
4. Agent calls the `end_call` tool with the appropriate reason

The agent does not end calls just because the caller said "thank you" or there was a pause. The master prompt explicitly instructs: "Let them decide when to end the call."

Farewell reason codes:

Reason	When Used
conversation_complete	Normal call completion
customer_requested_hangup	Caller explicitly said bye
order_confirmed	Food order was placed
booking_confirmed	Reservation was made

After calling `end_call`, the tool waits 1 second (grace period for final TTS audio to finish playing) and then deletes the LiveKit room, which immediately terminates the call.

Hold Messages and Wait Prompts

When the agent calls a tool (checking availability, creating a booking), it announces what it is doing before the tool executes. These announcements are defined in the master voice prompt configuration:

Tool	Announcement
check_availability	"Let me check that..."
create_booking	"Booking that now..."
get_business_hours	"Looking that up..."
transfer_to_human	"Connecting you now."
Default (all others)	"One moment..."

These announcements fill the silence during tool execution so the caller does not think the call dropped. The system prompt instructs the LLM to be specific: "Checking February 15th..." is preferred over "One moment please."

---

25. Agent Personality Tuning

Formality Levels

The personality tone is set per tenant and controls how the agent speaks. Four levels are available:

Level	Behavior	Example Phrase
casual	Relaxed, like a friend	"Hey, what's up?"
friendly	Warm but professional	"Hi there, how can I help?"
professional	Businesslike, polished	"Good afternoon."
formal	Proper etiquette, no slang	"Thank you for calling."

The tone is injected into the system prompt by the `buildSystemPrompt` function in `chat.ts`. Each tone value maps to a specific personality instruction that shapes the LLM's response style throughout the call.

Verbosity Control

Verbosity determines how much detail the agent provides in responses:

Level	Behavior	Best For
concise	1-2 sentences max	High call volume businesses
balanced	Enough detail to help	Most businesses
detailed	Thorough explanations	Complex services, medical

The master voice prompt enforces a baseline of short responses regardless of verbosity setting. Even "detailed" mode keeps responses to 2-3 sentences. The voice channel requires brevity – long responses cause callers to lose attention or interrupt.

Empathy Settings

Empathy controls how the agent responds to frustrated or upset callers:

Level	Behavior
high	Acknowledge emotions, validate, slower pace
medium	Acknowledge the situation, stay supportive
low	Focus on efficiency, minimal emotion

High empathy is recommended for medical and dental offices where callers may be anxious. Low empathy works for quick-service businesses where callers want speed over sympathy.

Humor and Personality Traits

The master voice prompt supports contextual humor through the `[laughter]` tag, but humor is used sparingly. The agent only laughs when the caller makes an actual joke. The prompt explicitly states: "NEVER force laughter. NEVER laugh at nothing."

Additional personality traits are shaped by:

- **Filler words**: "So…", "Let me see…", "Hmm…" used 1-2 times per conversation to sound human
- **Contractions**: Always uses "I'll", "we're", "that's" instead of formal forms
- **Energy matching**: Mirrors the caller's energy level (excited, frustrated, rushed, polite)
- **Active listening**: Uses "Mhmm", "Got it", "Right" to show engagement

These traits are defined in the master voice prompt and apply to all tenants. They cannot be overridden per tenant.

Industry-Appropriate Tone Examples

Industry	Recommended Tone	Verbosity	Empathy
Hotel	friendly	balanced	medium
Restaurant	casual	concise	medium
Medical	professional	balanced	high
Dental	professional	balanced	high
Salon	friendly	concise	medium
Auto service	professional	balanced	medium

A/B Testing Personality Variations

Currently, personality settings are tenant-wide and apply to all calls. To A/B test personality variations:

1. Create two tenant accounts with different personality settings
2. Assign different phone numbers to each account
3. Route a percentage of calls to each number using your SIP trunk provider
4. Compare metrics (booking rate, call duration, escalation rate) between the two
5. Apply the winning configuration to the primary account

Future releases may support built-in A/B testing with automatic traffic splitting and metric comparison.

---

26. Performance Optimization

End-to-End Latency Breakdown

The voice pipeline processes each conversation turn through multiple stages. Target latencies:

Stage	Component	Target
Speech detection	Silero VAD	< 50ms
Turn detection	Multilingual model	< 100ms
Speech-to-text	Deepgram Nova-3	200-400ms
LLM inference	GPT-4.1-mini	300-800ms
Text-to-speech	Cartesia Sonic-3	100-300ms
Network overhead	SIP + WebRTC	50-150ms
Total	**End-to-end**	**800-1800ms**

The target is for the caller to hear the first word of the agent's response within 1-2 seconds of finishing their sentence.

Prompt Caching

OpenAI's prompt caching significantly reduces LLM latency and cost. The system prompt (the same for every turn in a call) is cached after the first request.

How it works:

- First turn: Full system prompt sent and cached.
- Subsequent turns: Cache hit. Only new user message processed.
- Cache hit rate: ~88% in production.
- Cost reduction: Cached tokens cost 50% less.
- Latency reduction: Cached prompts skip re-tokenization.

The system prompt for a typical tenant is approximately 5,600 tokens. With caching, this is only processed fully once per call rather than on every turn.

Connection Pooling

The Python agent uses `httpx.AsyncClient` as a persistent HTTP client for API calls. This maintains a connection pool rather than opening new TCP connections per tool call. The client is initialized once via `get_client()` and reused across all tool invocations.

Reducing First-Response Time

The first response is the most latency-sensitive moment. Several optimizations reduce it:

1. **VAD prewarming**: Silero VAD loaded during process startup in `prewarm`, not on the first call.
2. **Preemptive generation**: `preemptive_generation=True` starts LLM before the caller finishes speaking.
3. **Greeting bypass**: Greeting generated via `on_enter()` immediately, without waiting for caller input.
4. **Streaming TTS**: Cartesia streams audio chunks; playback starts before full response is ready.

Audio Codec Optimization

The voice pipeline uses telephony-optimized settings:

Setting	Value	Reason
TTS speed	0.95	Slightly slower for clarity
TTS emotion	Content	Neutral warm baseline
Noise cancellation	BVCTelephony	Optimized for phone audio
Sample rate	8kHz (SIP default)	Standard telephony rate

The `BVCTelephony` noise cancellation plugin is designed for telephone audio, removing background noise and echo.

Monitoring Latency Metrics

The agent collects real-time metrics through LiveKit `UsageCollector`:

- LLM prompt and completion tokens per turn
- STT processing duration per utterance
- TTS generation duration per response
- Total session duration

These metrics are logged at the end of each call via `on_shutdown`. Monitor for:

- LLM latency spikes (cache misses or model slowdowns)
- STT processing time increases (audio quality issues)
- TTS generation delays (may need voice or model change)

---

27. Security Considerations

Call Recording Consent

Call recording is subject to state and federal laws:

Jurisdiction Type	Requirement
One-party consent (US federal)	One party must consent
Two-party consent (CA, FL, etc.)	All parties must consent
International	Varies by country

The Lumentra voice agent does not record calls as audio files. It stores transcripts (text) from the STT pipeline. Storing transcripts may also be subject to recording consent laws depending on jurisdiction.

Recommendations:

- Include a recording disclosure in your greeting if required
- Example: “This call may be recorded for quality purposes”
- Consult legal counsel for your specific jurisdiction
- Configure the greeting text to include disclosure if needed

PII Handling

The system stores personally identifiable information:

Data Type	Where Stored	Retention
Caller phone number	calls, contacts tables	Indefinite
Caller name	calls, contacts tables	Indefinite
Transcript text	calls table	Indefinite
Booking details	bookings table	Indefinite
Call summary	calls table	Indefinite

Transcripts may contain sensitive information spoken during calls. The system does not perform automatic PII redaction.

Recommendations:

- Implement transcript redaction for sensitive data
- Set up data retention policies to purge old transcripts
- Train the agent via custom instructions to never ask for credit card numbers or SSNs
- Review transcripts periodically for unintended PII

Data Retention Policies

The system does not enforce automatic retention limits by default. All records are stored indefinitely in Supabase PostgreSQL.

To implement data retention:

- Create a scheduled job to delete records older than your retention period
- Typical periods: 30 days for transcripts, 1 year for metadata
- Retention varies by industry (HIPAA: 6 years for medical)
- Consider archiving rather than deleting for compliance

Encryption

Layer	Method
API traffic (HTTPS)	TLS 1.2+ via reverse proxy
Database connections	SSL to Supabase PostgreSQL
SIP signaling	TLS (SignalWire)
Media (RTP)	SRTP (encrypted media)
WebRTC (LiveKit)	DTLS-SRTP (mandatory)
Data at rest	Supabase managed encryption

All voice media between caller and LiveKit is encrypted via SRTP. Agent-to-provider connections use HTTPS/WSS with TLS.

Access Control

Resource	Who Can Access
Call transcripts	Tenant admin via dashboard
Call recordings	Not implemented (no audio)
Agent configuration	Tenant admin via dashboard
Internal API	Agent process (bearer token)
Database	API service (Supabase role)
LiveKit rooms	Agent process (API key)

The internal API is authenticated with a bearer token (`INTERNAL_API_KEY`) stored as an environment variable, never exposed to the browser.

HIPAA Considerations for Medical Tenants

Medical tenants handling PHI should be aware:

Concern	Status
BAA with OpenAI (LLM)	Required before use
BAA with Deepgram (STT)	Required before use
BAA with Cartesia (TTS)	Required before use
PHI in transcripts	May be present

Concern	Status
PHI sent to LLM	Sent for processing
Audit logging	Call logs with timestamps

The medical prompt instructs HIPAA-compliant language and avoidance of medical details aloud. However, callers may disclose PHI. Regulated tenants must consult compliance teams.

Audit Logging

Every call generates an audit trail:

1. **Call start**: LiveKit room created, SIP participant joined
2. **Tenant lookup**: API logs phone number lookup
3. **Tool calls**: Each invocation logged with action and arguments
4. **Call completion**: Full record with transcript, duration, outcome
5. **Post-call automation**: CRM actions logged

Logs available through Supabase dashboard (Postgres logs) and agent container stdout (`docker logs`).

---

28. Troubleshooting Common Voice Issues

Echo and Audio Feedback

Symptoms: Caller hears their own voice echoed back, or the agent hears itself.

Cause	Fix
Missing noise cancellation	Verify BVCTelephony enabled
Caller on speakerphone	Cannot fix server-side
SIP codec mismatch	Check SignalWire codec config
Double audio tracks	Verify room has one SIP input

The agent uses `noise_cancellation.BVCTelephony()` in `RoomOptions`. If echo persists, check the noise cancellation plugin is installed in the Docker image.

High Latency: Diagnosing Slow Responses

Symptoms: Long pauses (3+ seconds) between caller finishing and agent responding.

Diagnostic steps:

1. **Check LLM latency**: Metrics logs for token generation time. If high, check prompt cache hit rate.
2. **Check STT latency**: Long processing indicates audio issues or network problems to Deepgram.
3. **Check turn detection**: If `min_endpointing_delay` is too high, agent waits too long. Default is 0.8s.
4. **Check network**: Verify latency to API providers (OpenAI, Deepgram, Cartesia).
5. **Check system load**: High CPU slows all pipeline stages.

Tuning parameters:

Parameter	Default	Effect
min_endpointing_delay	0.8s	Min wait after silence
max_endpointing_delay	2.5s	Max wait before turn end

Parameter	Default	Effect
false_interruption_timeout	1.5s	Wait before resuming
preemptive_generation	true	Start LLM early

Dropped Calls

Symptoms: Call disconnects mid-conversation.

Cause	Diagnostic	Fix
Network timeout	Check SIP trunk logs	Verify UDP/RTP ports
LiveKit room timeout	Check room duration	Increase room timeout
Agent crash	Check container logs	Fix error in tool code
SIP INVITE timeout	Check SignalWire	Verify SIP endpoint
Duration limit hit	Check watchdog logs	Increase max_call_duration

Check agent container logs first: `docker logs livekit-agent --tail 100`

Garbled Audio

Symptoms: Audio sounds distorted, choppy, or unintelligible.

Cause	Fix
Packet loss	Check network quality
Codec mismatch	Verify SIP trunk codec
Sample rate mismatch	Ensure 8kHz for telephony
Buffer underrun	Check server CPU load

Agent Not Answering

Symptoms: Call rings but no one answers, or caller hears silence.

Diagnostic checklist:

1. Is the agent container running? `docker ps`
2. Is the agent registered with LiveKit?
3. Is the SIP dispatch rule correct? `agent_name` must match “lumentra-voice-agent”.
4. Can the agent reach the API? Check connectivity to internal endpoint.
5. Is there a tenant for the dialed number? Check logs for “No tenant found”.

Caller Cannot Hear the Agent

Symptoms: Caller says “Hello?” but agent is responding (visible in logs).

Cause	Fix
TTS output not playing	Check Cartesia API key
Audio track not published	Check LiveKit room tracks
SIP media path broken	Verify RTP port range open
Cartesia rate limit	Check TTS error logs
Wrong voice ID	Verify voice_id in config

Agent Talks Over the Caller

Symptoms: Agent responds while caller is still speaking.

Parameter	Current	Adjustment
min_endpointing_delay	0.8s	Increase to 1.0-1.2s
max_endpointing_delay	2.5s	Increase to 3.0s
false_interruption_timeout	1.5s	Increase to 2.0s
resume_false_interruption	true	Keep enabled

Increasing `min_endpointing_delay` makes the agent wait longer before deciding the caller has stopped. The `resume_false_interruption` setting resumes the agent's response if the interruption was just noise.

---

29. Appendix A: Complete Configuration Reference

Personality Settings

Field	Type	Default	Valid Values
tone	string	"friendly"	casual, friendly, professional, formal
verbosity	string	"balanced"	concise, balanced, detailed
empathy	string	"medium"	low, medium, high

Voice Settings

Field	Type	Default	Valid Values
voice_id	string	(varies)	Cartesia voice UUID
tts_speed	number	0.95	0.5 - 2.0
tts_emotion	string	"Content"	Cartesia emotion values
stt_model	string	"nova-3"	Deepgram model names
stt_language	string	"multi"	Language codes or "multi"

Agent Identity

Field	Type	Default	Description
agent_name	string	"Alex"	Agent's spoken name
business_name	string	(required)	Business name in prompts
industry	string	(required)	Industry type identifier

Valid industry values: `hotel`, `motel`, `restaurant`, `medical`, `dental`, `salon`, `auto_service`.

Greeting Configuration

Field	Type	Default	Description
greeting_standard	string	(generated)	Default call greeting
greeting_after_hours	string	null	After-hours greeting
greeting_returning	string	null	Returning caller greeting

Operating Hours

Field	Type	Default	Description
schedule	array	[]	Array of day/open/close
schedule[].day	string	(required)	Day name (Monday, etc.)
schedule[].open	string	(required)	Opening time (HH:MM)
schedule[].close	string	(required)	Closing time (HH:MM)
holidays	array	[]	Holiday date strings
timezone	string	“America/New_York”	IANA timezone

Escalation Settings

Field	Type	Default	Description
escalation_enabled	boolean	false	Enable call transfers
escalation_phone	string	null	Transfer destination
escalation_triggers	array	[]	Trigger conditions

Call Duration

Field	Type	Default	Range
max_call_duration_seconds	number	900	120 - 3600

Turn Detection Tuning

Field	Type	Default	Range
min_endpointing_delay	number	0.8	0.3 - 2.0
max_endpointing_delay	number	2.5	1.0 - 5.0
false_interruption_timeout	number	1.5	0.5 - 3.0
preemptive_generation	boolean	true	true / false
resume_false_interruption	boolean	true	true / false

Tools Reference

Tool	Required Args	Optional Args
check_availability	date	service_type

Tool	Required Args	Optional Args
create_booking	customer_name, customer_phone, date, time	service_type, notes
create_order	customer_name, order_type, items	customer_phone, delivery_address, special_instructions
transfer_to_human	reason	(none)
end_call	reason	(none)
log_note	note	note_type

Tool Argument Details

transfer_to_human reason values: `customer_request`, `complaint`, `refund_request`, `cannot_resolve`

end_call reason values: `conversation_complete`, `customer_requested_hangup`, `order_confirmed`, `booking_confirmed`

log_note note_type values: `general`, `preference`, `complaint`, `compliment`, `follow_up`, `internal`

create_order order_type values: `pickup`, `delivery`

---

30. Appendix B: Sample System Prompts

The system prompt is assembled dynamically from multiple layers. Below are annotated examples for each supported industry.

Hotel System Prompt

```
[MASTER VOICE PROMPT - Platform Level]
You are speaking on a live phone call.
Your text output will be converted to speech.
=== HIGH-STAKES ACCURACY REQUIRED ===
[... voice format rules, SSML markers,
     turn-taking, name spelling, cultural
     awareness, confirmation verification ...]

---

[AGENT IDENTITY - Per Tenant]
You are Sarah, the receptionist at
Grand Plaza Hotel.

## Your Role
You help guests check room availability,
make reservations, and answer questions.
You ARE the front desk.

## Personality
- Be warm, friendly, and approachable
- Provide enough detail without being wordy
- Acknowledge the caller's situation

## Business Context
Industry: hotel
```

```
Business: Grand Plaza Hotel
Today's Date: 2026-03-02
Location: 123 Main St, Miami
Timezone: America/New_York

## Operating Hours
- Monday: 00:00 - 23:59
[... 24/7 for hotels ...]

Transfer phone: +15551234567

[INDUSTRY RULES - Hotel Specific]
## STATE TRACKING - CRITICAL
Track: check-in date, nights, guests,
room type, name.
NEVER re-ask for info you already have.

## BOOKING FLOW (flexible, natural)
1. Dates  2. Nights  3. Guests
4. Room   5. Availability
6. Name   7. Confirm and code

## QUICK ANSWERS
- Check-in 3 PM, checkout 11 AM
- Free parking, free WiFi
- Cancel free up to 24 hours before

[FINAL CRITICAL RULES - Platform Level]
- NEVER say you are an AI
- NEVER mention tools or functions
- Keep responses SHORT
- ONE question per response
- Sound HUMAN
```

Restaurant System Prompt

```
[MASTER VOICE PROMPT]
[... same platform rules ...]

---

You are Marco, the receptionist at
Bella Cucina.

## Your Role
You help guests make table reservations,
answer menu questions, provide info.

## Personality
- Keep things casual and relaxed
- Keep responses brief
- Be understanding

## Business Context
Industry: restaurant
Business: Bella Cucina
Location: 456 Oak Ave, Austin
```

```
Timezone: America/Chicago

## Operating Hours
- Monday: Closed
- Tuesday: 11:00 - 22:00
[...]

## STATE TRACKING - CRITICAL
Track: date, time, party size, name,
special occasion.

## RESERVATION BOOKING FLOW
1. Date and time
2. Party size
3. Check availability
4. Name (ALWAYS ask to spell)
5. Special occasion
6. Confirm ALL details back
7. Call create_booking - MANDATORY
8. Give confirmation code
9. Mention 15-min hold policy

## COMMON QUESTIONS
- Hours: Use operating hours above
- Dress code: Smart casual
- Private events: Take number, callback
```

Medical Office System Prompt

```
[MASTER VOICE PROMPT]
[... same platform rules ...]

---

You are Lisa, the receptionist at
Westside Family Medicine.

## Your Role
You help schedule appointments, handle
prescription refills, answer questions.

## Personality
- Maintain a professional demeanor
- Provide enough detail to be helpful
- Show strong empathy, validate concerns

## CRITICAL MEDICAL RULES
- NEVER provide medical advice
- NEVER discuss medications or dosages
- Emergencies: "Please call 911"
- Prescription refills: Take message
- Confirm full name and date of birth
- Use HIPAA-compliant language
- Test results: "Doctor will call back"

## APPOINTMENT BOOKING FLOW
1. New or existing patient?
```

```
2. Which provider? (if multiple)
3. Reason for visit (general only)
4. Check availability, offer times
5. Confirm: name, DOB, date, time
6. Remind: arrive 15 min early
```

Dental Office System Prompt

```
[MASTER VOICE PROMPT]
[... same platform rules ...]

---

You are Amy, the receptionist at
Bright Smile Dental.

## Your Role
You help schedule dental appointments
and answer service questions.

## Personality
- Professional demeanor
- Enough detail to be helpful
- Show strong empathy

## CRITICAL DENTAL RULES
- NEVER provide dental advice
- Emergencies: offer same-day appointment
- Confirm name and date of birth
- Costs vary by insurance

## APPOINTMENT BOOKING FLOW
1. Routine cleaning or specific concern?
2. New or existing patient?
3. Check availability, offer times
4. Confirm: name, date, time
5. Remind: arrive 10 min early
```

Salon System Prompt

```
[MASTER VOICE PROMPT]
[... same platform rules ...]

---

You are Jessica, the receptionist at
Luxe Salon.

## Your Role
You help schedule appointments and
answer service questions.

## Personality
- Warm, friendly, approachable
- Keep responses brief
- Be understanding
```

```
## SALON BOOKING RULES
- Ask service (haircut, color, nails)
- Ask preferred stylist
- Mention service duration
- Note special requests

## APPOINTMENT BOOKING FLOW
1. What service?
2. Preferred stylist or next available?
3. Check availability, offer times
4. Name for the appointment?
5. Confirm: name, service, stylist, time
6. Remind: arrive 5-10 min early
```

Auto Service System Prompt

```
[MASTER VOICE PROMPT]
[... same platform rules ...]

---

You are Mike, the receptionist at
Precision Auto Care.

## Your Role
You help schedule service appointments
and answer service questions.

## Personality
- Professional demeanor
- Enough detail to be helpful
- Be understanding

## AUTO SERVICE RULES
- Ask what service needed
- Get vehicle year, make, model
- Note symptoms or warning lights
- Mention estimated service duration

## SERVICE APPOINTMENT FLOW
1. What service?
2. Vehicle year, make, model?
3. Concerns or warning lights?
4. Check availability, offer times
5. Name for the appointment?
6. Confirm: name, vehicle, service, time
7. Remind: bring keys and paperwork
```

---

31. Appendix C: Glossary of Voice AI Terms

Audio and Telephony

Term	Definition
SIP	Session Initiation Protocol for calls over IP

Term	Definition
SIP REFER	SIP method for call transfer
SIP Trunk	Virtual phone line to the phone network
RTP	Real-time Transport Protocol for audio
SRTP	Secure RTP with encryption
WebRTC	Web Real-Time Communication standard
DTLS-SRTP	Security protocol for WebRTC media
DTMF	Dual-Tone Multi-Frequency (touch-tones)
Codec	Audio compression (OPUS, PCMU, PCMA)
Sample rate	Audio samples/sec (8kHz telephony)
PSTN	Public Switched Telephone Network

Speech Processing

Term	Definition
STT	Speech-to-Text (audio to text)
TTS	Text-to-Speech (text to audio)
VAD	Voice Activity Detection (speech vs silence)
Endpointing	Deciding when a speaker finished talking
Barge-in	Caller interrupting agent mid-sentence
Turn detection	Determining whose turn to speak
Diarization	Identifying which speaker said what
SSML	Speech Synthesis Markup Language
Streaming STT	Real-time audio processing
Streaming TTS	Audio generation with early playback
Smart formatting	Auto-format numbers, dates in STT

Language Models

Term	Definition
LLM	Large Language Model for text generation
Prompt caching	Reusing processed prompt tokens
Token	Unit of text (~4 chars in English)
Context window	Max tokens per request (128k)
System prompt	Instructions before conversation
Temperature	Controls output randomness (0-1)
Function calling	LLM invoking tools with arguments
Inference	Generating output from a model
Completion tokens	Output tokens from the LLM
Prompt tokens	Input tokens to the LLM

LiveKit Concepts

Term	Definition
Room	A LiveKit room; one per phone call
Participant	Entity in a room (caller, agent)
Track	Single audio or video stream
SIP participant	Caller connected via SIP bridge
Agent session	Runtime instance for one call
Room deletion	Ending a call by destroying room

Term	Definition
Prewarm	Loading models at startup
Usage collector	Collects per-call metrics
Dispatch rule	Routes calls to agent names

Platform Concepts

Term	Definition
Tenant	A business account in the platform
Multi-tenant	One system serving many businesses
Pipeline	CRM stages for deals/bookings
Escalation	Transferring call to human
Outcome	Classification of call result
Post-call automation	CRM actions after call ends
Confirmation code	6-char alphanumeric for bookings
Contact	Caller record with phone and history
Custom instructions	Business-specific prompt knowledge
Master voice prompt	Platform speech rules (immutable)

---

32. Appendix D: Frequently Asked Questions

Q1: How quickly does the agent answer after setup? A: Immediately once the phone number and SIP trunk are active. The first call after a container restart may have 1-2 seconds extra latency from model loading.

Q2: Can I change the agent's voice? A: Yes. Settings > Assistant, select a different voice. Takes effect on the next call. In-progress calls are unaffected.

Q3: What happens when the agent cannot understand? A: The agent asks to repeat: "Sorry, what was that?" It never comments on accents. After multiple failures, it offers a transfer to a human.

Q4: Can the agent handle multiple languages? A: Deepgram Nova-3 recognizes 36+ languages in multi mode. Agent responses are English by default. Other response languages require custom prompt configuration.

Q5: How much does each call cost? A: About $0.015 per call with GPT-4.1-mini (2-3 min average, 88% cache hit rate). Covers STT, LLM, and TTS. SIP trunk costs are separate.

Q6: What is the maximum call duration? A: Default 900 seconds (15 min), minimum 120 seconds (2 min). Configurable per tenant. Agent wraps up and offers human transfer as the limit approaches.

Q7: Can the agent transfer to departments? A: Currently one escalation phone per tenant. Department routing is not yet supported.

Q8: What if the internet goes down mid-call? A: The call drops. All services (SIP, LiveKit, API) must be reachable. The call is logged if the shutdown callback fires first.

Q9: Can I review call transcripts? A: Yes. Transcripts are in the database and accessible through the dashboard with full conversation text, duration, outcome, and summary.

Q10: How do I add custom knowledge? A: Settings > Instructions. Add menu items, pricing, policies as free-form text. This becomes part of the system prompt.

Q11: Why does the agent sound robotic? A: Check personality: use "friendly" or "casual" tone. Set verbosity to "concise" or "balanced". Detailed responses sound scripted when spoken.

Q12: Can the agent make outbound calls? A: No. Inbound only. Outbound requires additional SIP and agent configuration not currently available.

Q13: How do I test without real calls? A: Call your configured number directly. No browser test mode exists. Modify settings and observe on the next call.

Q14: What happens after hours? A: Agent uses after-hours greeting if configured and informs callers of business hours. The agent still answers – no voicemail by default.

Q15: Can it handle simultaneous calls? A: Yes. Each call gets its own LiveKit room and agent instance. Capacity is limited by server resources.

Q16: How is caller identity determined? A: Phone number extracted from SIP attributes (`sip.phoneNumber`). Contact record created or found by phone. Caller ID name is not available.

Q17: Can I disable specific tools? A: Capabilities configuration controls tool access. The system prompt includes or excludes instructions for each capability.

Q18: What is the confirmation code format? A: 6-character alphanumeric (e.g., A3K7P2). Read back with the SSML spell tag for clarity.

Q19: How do I change the agent's name? A: Settings > Assistant, update agent name. Appears in the prompt as "You are {name}, the receptionist at {business}." Next call picks it up.

Q20: What if caller hangs up mid-booking? A: No booking created if `create_booking` was not called. Transcript captured up to disconnection. Outcome classified from transcript content.

Q21: Can I see real-time call activity? A: Active calls visible in LiveKit dashboard (rooms and participants). Lumentra dashboard shows history after completion. Live monitoring not yet supported.

Q22: Do settings changes require a restart? A: No. Tenant config is fetched fresh from the API at the start of each call. No container restart needed.

Q23: What STT model is used? A: Deepgram Nova-3 with multi-language and smart formatting. Set in agent code, not configurable per tenant via dashboard.

Q24: How does it handle background noise? A: BVCTelephony noise cancellation removes background noise, echo, and feedback. Silero VAD distinguishes speech from noise before STT.